Supplementary proof manual for the string-flow proof of unboundedness of $5x + 1$ at 7

StringFlow

August 20, 2026

Abstract

This manual is the companion to the main paper. It records the full proof chain, the Lean theorem index, and the dependency ledger. The current compiled length is 102 pages; the target length is 100 to 150 pages, and the chapters are maintained incrementally.

Contents

1 String-flow vocabulary

This chapter records the exact vocabulary used in the main paper and in the Lean repository.

1.1 Words

A word is a finite list of natural numbers $t_0, \ldots, t_{L-1}$. The length is L. The word is valid from x_0 when each division

$$x_{i+1} = \frac{5x_i + 1}{2^{t_i}}$$

is exact.

The Lean names are:

- `StringFlow.Word.wordValid`;
- `StringFlow.Word.wordOrbit`;
- `StringFlow.wordWeight`;
- `StringFlow.Word.wordA`.

1.2 Recursive definitions

The Lean definitions in `WordWindow.lean` are:

```
def wordA : List Nat -> Nat
  | [] => 0
  | t :: ts => 5 ^ ts.length + 2 ^ t * wordA ts

def wordOrbit : List Nat -> Nat -> Nat
  | [], x => x
  | t :: ts, x => wordOrbit ts ((5 * x + 1) / 2 ^ t)

def wordValid : List Nat -> Nat -> Prop
  | [], _ => True
  | t :: ts, x => (5 * x + 1) % 2 ^ t = 0 /\
    wordValid ts ((5 * x + 1) / 2 ^ t)
```

The paper writes these as A, wordOrbit, and the validity predicate defined in the main text.

1.3 The valuation lemma

Every positive integer n decomposes uniquely as

$$n = 2^{v_2(n)}\ \mathrm{oddpart}(n), \qquad \mathrm{oddpart}(n) \equiv 1 \pmod 2.$$

The exact step weight is the maximal legal weight:

$$t = v_2(5x + 1)\,,$$

while `wordValid` only requires divisibility by 2^t. A legal word may therefore use a smaller weight, which is why GeneralOrbitFrom7 can reach intermediate states not on the accelerated orbit. The Lean names are `n_eq_two_pow_mul_oddPart` and `oddPart_odd_of_pos`.

1.4 Word molecule identity

For a valid word w from x_0,

$$2^{W(w)}\ \mathrm{wordOrbit}(w, x_0) = 5^{\mathrm{len}(w)}\ x_0 + A(w).$$

1.5 Proof of the word orbit identity

Proceed by induction on w. For $w = []$, both sides equal x_0. For $w' = t :: w$, the inductive hypothesis applied to $(5x_0 + 1)/2^t$ gives

$$2^{W(w)}\,\mathrm{wordOrbit}\left(w, \frac{5x_0+1}{2^t}\right) = 5^{\mathrm{len}(w)}\frac{5x_0+1}{2^t} + A(w).$$

Multiplying by 2^t and using $W(w') = t + W(w)$ and $A(w') = 5^{\mathrm{len}(w)} + 2^t A(w)$ gives the identity.

1.6 Exact orbit bound

For $n \geq 2$,

$$\mathrm{Orbit}_7(n) < 5^n.$$

If $x < 5^k$, then $5x + 1 < 2 \cdot 5^{k+1}$, so $(5x + 1)/2 < 5^{k+1}$; since the accelerated step is at most $(5x + 1)/2$, induction gives the bound. The Lean statement is `fiveXPlusOneOrbit_lt_five_pow`.

1.7 PMI identity

With $m_j = j \log_2 5 - W_j$,

$$\sum_{j=0}^{L-1} 2^{-m_j} = \frac{5A(w)}{5^L}.$$

The cleared-denominator form is

$$\sum_{j=0}^{L-1} 2^{W_j}\, 5^{L-j} = 5A(w).$$

For a closed cycle word of period P and cycle state m, the Lean form is

$$\mathrm{aTotal5}(P, t) = 5m(2^T - 5^P), \qquad T = W(w),$$

proved in `StringFlow.PMI.pmi_algebraic`.

1.8 PMI-B

The PMI-B layer compares bad-prefix counts with the available budget. For a closed cycle word, the total weight satisfies

$$2^{W(w)}\, m = 5^P\, m + A(w).$$

1.9 PMI-B derivation

Write B for the number of bad proper prefixes. The $j = 0$ term of aTotal5 contributes 5^P, and each bad prefix contributes at least another 5^P, so

$$5^P + B \cdot 5^P \leq \mathrm{aTotal5}(P, t).$$

The cycle equation gives

$$\mathrm{aTotal5}(P, t) = 5m(2^T - 5^P),$$

hence

$$5^P + B \cdot 5^P \leq 5m(2^T - 5^P).$$

If the right-hand side is below $2 \cdot 5^P$, then $B = 0$ and every proper prefix satisfies $2^{W_j} < 5^j$. These are `pmi_b_count` and `pmi_b_no_bad_prefix`.

1.10 Margin balance

Let C_1, C_2, C_3 be the counts of steps of weight 1, 2, and at least 3. The nonnegative-margin condition gives

$$(3-\alpha)C_3 \le (\alpha-1)C_1 + (\alpha-2)C_2, \qquad \alpha = \log_2 5,$$

with the strict version used for the small-budget cycle words.

1.11 Rise steps and residues modulo 5

A rise step of weight 1 satisfies

$$2y = 5x + 1,$$

so $y \equiv 3 \pmod 5$. A rise step of weight 2 satisfies

$$4y = 5x + 1,$$

so $y \equiv 4 \pmod 5$. These rules are formalised as `rise_step_next_mod_five` and feed the block-head residue lemma of the cycle bridge.

1.12 Derivation of the PMI identity

Write the word molecule as

$$A(w) = \sum_{j=0}^{L-1} 2^{W_j}\, 5^{L-1-j}.$$

Divide $5A(w)$ by 5^L:

$$\frac{5A(w)}{5^L} = \sum_{j=0}^{L-1} 2^{W_j}\, 5^{-j}.$$

Since

$$2^{-m_j} = 2^{W_j - j\log_2 5} = 2^{W_j}\, 5^{-j},$$

the identity follows.

The same derivation for a closed cycle word with cycle state m and period P gives the algebraic form

$$\mathrm{aTotal5}(P,t) = 5m(2^T - 5^P),$$

where $T = W(w)$ is the total cycle weight.

1.13 Worked example

The word $[2,1]$ is valid from 7, with

$$7 \xrightarrow{t=2} 9 \xrightarrow{t=1} 23,$$

so $\mathrm{wordOrbit}([2,1],7) = 23$. Its word molecule is

$$A([2,1]) = 2^0 \cdot 5^1 + 2^2 \cdot 5^0 = 9,$$

and

$$2^3 \cdot 23 = 184 = 5^2 \cdot 7 + 9.$$

The word $[1,2]$ is valid from 9:

$$9 \xrightarrow{t=1} 23 \xrightarrow{t=2} 29, \qquad A([1,2]) = 2^0 \cdot 5^1 + 2^1 \cdot 5^0 = 7,$$

with $2^3 \cdot 29 = 232 = 5^2 \cdot 9 + 7$.

The word $[1,1]$ is valid from 9 in the divisibility sense but is not the exact accelerated word: the second weight at 23 is $v_2(116) = 2$, so $\mathrm{wordOrbit}([1,1], 9) = 58$ is an intermediate state not on the accelerated orbit of 7. This is the distinction between GeneralOrbitFrom7 and FullOrbitFrom7.

1.14 Worked PMI example

For $w = [2,1,2]$ from 7, we have $L = 3$, $W = 5$, and $A(w) = 53$, with

$$2^5 \cdot 29 = 928 = 5^3 \cdot 7 + 53.$$

The prefix weights are $W_0 = 0$, $W_1 = 2$, $W_2 = 3$, so

$$\sum_{j=0}^{2} 2^{-m_j} = 1 + \frac{4}{5} + \frac{8}{25} = \frac{53}{25} = \frac{5A(w)}{5^3}.$$

1.15 Valuation toolkit

The following elementary facts are used throughout the proof:

- $n = 2^{v_2(n)}\,\mathrm{oddpart}(n)$ with odd part odd;
- if u is odd, then $v_2\big(2^k u\big) = k$;
- for odd x, $v_2(5x+1) \geq 1$;
- $T(x) \leq (5x+1)/2$;
- a C3 step from $x \geq 1$ satisfies $(5x+1)/2^t < x$.

The corresponding Lean names are listed in the main paper.

1.16 Notation summary

Symbol	Meaning
j	reset step depth
k_0	exponent in $s5^{k_0} = r + 1$
t	reset step weight, $t \in \{1,2\}$
δ	reset parameter: 1 for $t = 1$, $\{1,3\}$ for $t = 2$
s	odd terminal part with $5 \nmid s$
e	incoming step weight, $e \geq 2$
g	full-orbit state $\mathrm{Orbit}_7(j-1)$
x	candidate predecessor $2^{e-1}g + \delta 5^{j-1}$
r_j	block head after the reset step
r_s	terminal state of the block tail
t_j	step weight at depth j
W_j	prefix weight at depth j
L	block-tail length
H_s	unified-core threshold parameter

1.17 Block equation

For a block with step weights $u_1, \dots, u_L$ and total weight U,

$$2^U r_L = 5^L r_0 + B,$$

where B is the block molecule. The reset block satisfies

$$t = 2: \quad 4(r_j + 1) = 5^{k_0+1} s + \delta 5^j,$$

$$t = 1: \quad 2(r_j + 1) = 5^{k_0+1} s + 5^j - 2.$$

These equations are used by the corrected window bound. The block equation is exactly the word orbit identity applied to the block word; no averaging or approximation is involved.

For example, the block starting at 29 with weights $[1, 1, 2]$ has $L = 3$, $U = 4$, and $B = 39$, with

$$2^4 \cdot 229 = 3664 = 5^3 \cdot 29 + 39.$$

2 PMI and PMI-B budgets

This chapter develops the PMI identity and the PMI-B budget layer in full. The layer converts the exact word-orbit identity into counting inequalities that constrain the step weights of a closed cycle word.

2.1 The margin

For a word $w = (t_0, \dots, t_{L-1})$ with prefix weights W_j, define the margin after j steps by

$$m_j = j \log_2 5 - W_j.$$

The margin measures how much 2-adic budget remains after j exact steps. A prefix with $m_j \geq 0$ is called a *nonnegative-margin prefix*; a proper prefix with $m_j < 0$ is called a *bad prefix*.

2.2 The PMI identity

Theorem 2.1 (PMI identity)**.** *For every valid word w of length L,*

$$\sum_{j=0}^{L-1} 2^{-m_j} = \frac{5\, A(w)}{5^L}.$$

Proof. Write the word molecule as

$$A(w) = \sum_{j=0}^{L-1} 2^{W_j}\, 5^{L-1-j}.$$

Then

$$\frac{5\, A(w)}{5^L} = \sum_{j=0}^{L-1} 2^{W_j}\, 5^{-j}.$$

Since

$$2^{-m_j} = 2^{W_j - j \log_2 5} = 2^{W_j}\, 5^{-j},$$

the identity follows. The cleared-denominator form is

$$\sum_{j=0}^{L-1} 2^{W_j}\, 5^{L-j} = 5\, A(w).$$

□

2.3 The cycle form

Let w be a closed cycle word of period P with cycle state m, total weight $T = W(w)$, and prefix weights W_j. Define

$$\mathrm{aTotal5}(P, t) = \sum_{j=0}^{P-1} 2^{W_j}\, 5^{P-j}.$$

The closedness

$$\mathrm{wordOrbit}(w, m) = m$$

gives the cycle equation

$$2^T\, m = 5^P\, m + A(w).$$

Substituting into the cleared form yields

$$\mathrm{aTotal5}(P, t) = 5m\left(2^T - 5^P\right).$$

This is the algebraic form `StringFlow.PMI.pmi_algebraic` in Lean.

2.4 The counting bound

Write B for the number of bad proper prefixes of a closed cycle word of period P. The $j = 0$ term of aTotal5 contributes

$$2^{W_0}\, 5^P = 5^P.$$

Every bad prefix j satisfies

$$W_j > j \log_2 5,$$

equivalently

$$2^{W_j} > 5^j,$$

so its contribution satisfies

$$2^{W_j}\, 5^{P-j} > 5^P.$$

Combining the $j = 0$ term with the bad-prefix contributions gives

$$5^P + B \cdot 5^P < \mathrm{aTotal5}(P, t).$$

The paper writes this with $\leq$; the strict form follows because bad prefixes are defined by strict margin violation. Using the cycle form,

$$5^P + B \cdot 5^P < 5m\left(2^T - 5^P\right).$$

2.5 Small budget and no bad prefix

Corollary 2.2. *If*

$$5m\left(2^T - 5^P\right) < 2 \cdot 5^P,$$

then $B = 0$: every proper prefix satisfies

$$2^{W_j} < 5^j.$$

Proof. The counting bound gives

$$B \cdot 5^P < 5m\left(2^T - 5^P\right) - 5^P < 5^P,$$

so $B < 1$, hence $B = 0$. □

2.6 Margin balance

Let C_1, C_2, C_3 be the numbers of steps of weight 1, weight 2, and weight at least 3 in a word, and put $\alpha = \log_2 5$. The nonnegative-margin prefix condition implies

$$(3 - \alpha)C_3 \le (\alpha - 1)C_1 + (\alpha - 2)C_2.$$

The strict version, used for closed cycle words, replaces $\le$ by $<$.

The balance records that a C3 step spends at least $3 - \alpha$ units of margin, while a weight-1 step earns $\alpha - 1$ and a weight-2 step earns $\alpha - 2$. The proof telescopes the margin differences along the word and compares the final margin with the total weight. The Lean statements are `cycleWord_margin_balance` and `cycleWord_margin_balance_strict`.

2.7 Worked example: the exact prefix word

For the exact prefix word

$$w = [2, 1, 2]$$

from 7, we have $L = 3$, total weight $T = 5$, and $A(w) = 53$, with

$$2^5 \cdot 29 = 928 = 5^3 \cdot 7 + 53.$$

The prefix weights are

$$W_0 = 0, \quad W_1 = 2, \quad W_2 = 3,$$

so the margins are

$$m_0 = 0, \qquad m_1 = \log_2 5 - 2 \approx 0.3219, \qquad m_2 = 2\log_2 5 - 3 \approx 1.6439.$$

All proper prefixes are nonnegative, and the PMI sum is

$$\sum_{j=0}^{2} 2^{-m_j} = 1 + \frac{4}{5} + \frac{8}{25} = \frac{53}{25} = \frac{5A(w)}{5^3}.$$

2.8 Worked example: a bad prefix

The word $[3,1]$ has prefix weights

$$W_0 = 0, \quad W_1 = 3, \quad W_2 = 4,$$

and molecule

$$A([3,1]) = 2^0 \cdot 5^1 + 2^3 \cdot 5^0 = 13.$$

The margins are

$$m_0 = 0, \qquad m_1 = \log_2 5 - 3 \approx -0.6781, \qquad m_2 = 2\log_2 5 - 4 \approx 0.6439.$$

The proper prefix of length 1 is bad, so $B = 1$. The cleared sum is

$$\mathrm{aTotal5}(2,t) = 2^0 \cdot 5^2 + 2^3 \cdot 5^1 = 25 + 40 = 65 = 5A([3,1]).$$

The counting bound

$$5^2 + B \cdot 5^2 = 25 + 25 = 50 < 65$$

holds strictly. This example is the sharpest elementary illustration of the PMI-B counting mechanism: one bad prefix consumes one extra unit of 5^P, and the algebra keeps the exact count.

2.9 Why the budget is exact

No probabilistic replacement is involved. The margin is defined by the exact prefix weights of the word, and every inequality in this chapter is derived from the identity

$$2^{W_j}\, 5^{L-j} = 5^{L-m_j}.$$

The constants α, $3-\alpha$, and the threshold $2 \cdot 5^P$ appear because the word algebra is exact, not because of a statistical model.

2.10 Lean statements

- `StringFlow.PMI.aTotal5`: the cleared sum.
- the identity aTotal5 $= 5\,$aTotal in `Pmi.lean`: the algebraic clearing.
- `StringFlow.PMI.pmi_algebraic`: the cycle form $\mathrm{aTotal5}(P,t) = 5m(2^T - 5^P)$.
- `pmi_b_count`: the counting bound.
- `pmi_b_no_bad_prefix`: the small-budget corollary.
- `cycleWord_margin_balance` and `cycleWord_margin_balance_strict`: the margin balance.

2.11 Status

Statement	Status
PMI identity	Lean: compiled
cycle form	Lean: compiled
PMI-B counting bound	Lean: compiled
small-budget corollary	Lean: compiled
margin balance	Lean: compiled

3 Block calculus and reset equations

This chapter develops the block calculus used by the unified core and the corrected window bound. It records the small-block formulas, the reset equations, the first-block terminal algebra, and the block-head identities.

3.1 Block molecule formulas

For a block with step weights $u_1, \ldots, u_L$ and total weight

$$U = u_1 + \cdots + u_L,$$

the block molecule is the word molecule of the block word. The recursive definition gives the following closed forms.

block word	molecule B	total weight U
$[u_1]$	1	u_1
$[u_1, u_2]$	$5 + 2^{u_1}$	$u_1 + u_2$
$[u_1, u_2, u_3]$	$25 + 5 \cdot 2^{u_1} + 2^{u_1+u_2}$	$u_1 + u_2 + u_3$

The block equation is

$$2^U r_L = 5^L r_0 + B.$$

It is the exact word-orbit identity applied to the block word; no approximation is involved.

3.2 Derivation of the formulas

For a singleton word, $A([u_1]) = 1$ and $U = u_1$. For length two,

$$A([u_1, u_2]) = 2^0 \cdot 5^1 + 2^{u_1} \cdot 5^0 = 5 + 2^{u_1}.$$

For length three,

$$A([u_1, u_2, u_3]) = 2^0 \cdot 5^2 + 2^{u_1} \cdot 5^1 + 2^{u_1+u_2} \cdot 5^0 = 25 + 5 \cdot 2^{u_1} + 2^{u_1+u_2}.$$

These are the exact forms substituted into the segment equations in the unified-core chapter.

3.3 Worked block examples

The block starting at 7 with weights $[2, 1]$ has molecule 9:

$$2^3 \cdot 23 = 184 = 5^2 \cdot 7 + 9.$$

The block starting at 9 with weights $[1, 2]$ has molecule 7:

$$2^3 \cdot 29 = 232 = 5^2 \cdot 9 + 7.$$

The block starting at 29 with weights $[1, 1, 2]$ has molecule

$$B = 25 + 5 \cdot 2 + 2^{1+1} = 39,$$

and

$$2^4 \cdot 229 = 3664 = 5^3 \cdot 29 + 39.$$

3.4 Reset equations

A reset step of weight $t \in \{1, 2\}$ maps the candidate predecessor x to the block head r_j:

$$2^t\, r_j = 5x + 1.$$

The reset predecessor has the form

$$x = 5^{k_0}\, s + \delta\, 5^{j-1} - 1,$$

where $s5^{k_0} = r + 1$ for the previous terminal state r, and $\delta = 1$ for $t = 1$, $\delta \in \{1, 3\}$ for $t = 2$.

Substituting into $2^t r_j = 5x + 1$ gives

$$2^t\, r_j = 5^{k_0+1} s + \delta 5^j - 4.$$

Adding 2^t to both sides:

- for $t = 2$,
$$4(r_j + 1) = 5^{k_0+1} s + \delta 5^j;$$
- for $t = 1$,
$$2(r_j + 1) = 5^{k_0+1} s + 5^j - 2.$$

These are the exact reset equations encoded by `S6Audit.ResetHeadEq` and used in `reset_head_predecessor`.

3.5 First-block terminal algebra

Let g_{prev} be the state before the incoming step and let $e \geq 2$ be its weight:

$$5g_{\text{prev}} + 1 = 2^e\, g.$$

The first-block terminal state r is

$$r = \frac{5g_{\text{prev}} + 1}{2} = 2^{e-1} g.$$

With $k_0 = 0$, the previous terminal becomes

$$s = 2^{e-1} g + 1,$$

and the reset predecessor becomes the candidate

$$x = 2^{e-1} g + \delta 5^{j-1}.$$

This is `first_block_terminal_eq` and `candidateX_of_reset_and_terminal`.

3.6 Block-head identities

The block-head numerator has the exact identity

$$A_j + 5^j q = 2^L \left(B + \delta 5^j\right), \qquad 0 < B < 5^j, \quad A_j < 5^j.$$

Since $A_j < 5^j$, the quotient structure forces

$$q = m + \delta 2^L, \qquad m < 2^L.$$

This is `reset_q0_form`.

Under the reset equation, the same numerator has the block-head form

$$A_j + 5^j q = 2^{W_p} \left(5^{k_0+1} s - 4 + \delta 5^j\right).$$

This is `block_head_identity_of_reset`.

3.7 The q0 interval

The interval condition

$$2^{W_p} \le q < 2^{W_j}$$

is equivalent to the block-head bounds

$$5^j \le 2^t r_j, \qquad r_j < 5^j,$$

by `q0_interval_iff_rj_bound`. The two forms are used in different layers: the first belongs to the unified-core ledger, and the second belongs to the reset-window predicate.

3.8 Segment equations

For a full-orbit segment of length d from $g = \mathrm{Orbit}_7(j-1)$ to $x = \mathrm{Orbit}_7(j-1+d)$ with weights $u_1, \ldots, u_d$ and prefix weights W_i, the exact segment equation is

$$2^{W+e-1} g + 2^W \delta 5^{j-1} = 5^d g + \sum_{i=0}^{d-1} 2^{W_i} 5^{d-1-i},$$

where $W = W_d$ is the segment total weight and e is the incoming step weight. The derivation is the word-orbit identity

$$2^W x = 5^d g + A(w)$$

with the substitution $x = 2^{e-1} g + \delta 5^{j-1}$. The small-$d$ closed forms above are exactly what the exclusions use.

3.9 Residue rules for rise steps

A rise step of weight 1 satisfies

$$2y = 5x + 1,$$

so $y \equiv 3 \pmod 5$. A rise step of weight 2 satisfies

$$4y = 5x + 1,$$

so $y \equiv 4 \pmod 5$. These are the only residue constraints used at the block level; they are formalised in `rise_step_next_mod_five`.

3.10 Corrected reachability predicate

The corrected reachability predicate $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$ carries:

- the previous-terminal equation $s5^{k_0} = r + 1$;
- the general-orbit reachability $\mathrm{GeneralOrbitFrom7}(r)$;
- the full-orbit reachability $\mathrm{FullOrbitFrom7}(r_j)$;
- the exact reset equation $\mathrm{ResetHeadEq}(s, j, k_0, t, \delta, r_j)$;
- the size bound $s < 5^{j-1-k_0}$, the oddness of s, and $5 \nmid s$.

The predicate fixes the same j, k_0, t in all fields. A witness that only fixes the difference $j - k_0 - 1$ is weaker and is not sufficient for the corrected window bound.

3.11 Arithmetic counterexample

The unconstrained window bound is false. The witness

$$j = 36,\ k_0 = 0,\ t = 2,\ \delta = 3,\ s = 2^{83} - 3 \cdot 5^{35}$$

satisfies

$$5s + 3 \cdot 5^{36} = 5(2^{83} - 3 \cdot 5^{35}) + 3 \cdot 5^{36} = 5 \cdot 2^{83},$$

so the valuation is 83, while the unconstrained bound claims at most 82. The witness satisfies the size, parity, and coprimality conditions but fails the reachability fields of the corrected predicate. This is why the corrected bound is a projection of the unified core rather than a standalone arithmetic estimate.

3.12 Status

Statement	Status
small-block molecule formulas	Math: proven
reset equations	Lean: compiled
first-block terminal equation	Lean: compiled
candidate parameterisation	Lean: compiled
q0 interval equivalence	Lean: compiled
block-head identity of reset	Lean: compiled
corrected reachability definition	Lean: compiled

4 Unified core proof chain

This chapter expands the unified core proof of documents 36.30.22 and 36.30.23.

4.1 Candidate parameterisation

The block-head state is

$$r = \frac{A_j + 5^j q}{2^{W_j}}.$$

The predecessor candidate is

$$x = 2^{e-1} g + \delta 5^{j-1}, \qquad g = \mathrm{Orbit}_7(j-1).$$

Status:

- Math: proven.
- Lean: `reset_head_predecessor`, `candidateX_of_reset_and_terminal`, `first_block_terminal_eq` compiled.

4.2 Reset-head predecessor

Lemma 4.1. *If $1 \le j$, the reset equation* $\mathrm{ResetHeadEq}(s, j, k, t, \delta, r_j)$ *holds, and*

$$r_j = \frac{5x + 1}{2^t},$$

then

$$x = 5^k\, s + \delta\, 5^{j-1} - 1.$$

This is `reset_head_predecessor`.

4.3 First-block terminal equation

Lemma 4.2. *If* $5g_{\mathrm{prev}} + 1 = 2^e g$ *and*

$$r = \frac{5g_{\mathrm{prev}} + 1}{2},$$

then

$$r = 2^{e-1} g.$$

With $k = 0$ *and* $s = 2^{e-1}g + 1$*, the reset predecessor becomes*

$$x = \texttt{candidateX}\ j\ e\ g\ \delta.$$

The Lean names are `first_block_terminal_eq` *and* `candidateX_of_reset_and_terminal`*.*

4.4 Reset q0 form

Lemma 4.3. *From the exact identity*

$$A_j + 5^j q = 2^L (B + \delta 5^j),$$

with $0 < B < 5^j$ *and* $A_j < 5^j$*, there exists* $m < 2^L$ *with*

$$q = m + \delta\, 2^L.$$

This is `reset_q0_form`*.*

4.5 Block-head identity

Lemma 4.4. *Under the block-head representation and the reset equation,*

$$A_j + 5^j q = 2^{W_p} \left(5^{k+1} s - 4 + \delta 5^j\right).$$

This is `block_head_identity_of_reset`*.*

4.6 Orbit segment word

For d steps starting at depth $j - 1$, the segment word satisfies

$$2^W x = 5^d\, g + A(w),$$

where W is the total segment weight. This is `orbitSegmentWord_equation`. The orbit-level exclusions `d1_exclusion_of_orbit` and `d2_exclusion_of_orbit` are compiled.

4.7 Derivation of the segment equation

Let the segment word w have length d, total weight $W = W_d$, and prefix weights $W_0, \ldots, W_d$. The exact word-orbit identity applied to the segment from g to x gives

$$2^W x = 5^d\, g + A(w),$$

with

$$A(w) = \sum_{i=0}^{d-1} 2^{W_i}\, 5^{d-1-i}.$$

Substituting the candidate

$$x = 2^{e-1}g + \delta 5^{j-1}$$

and expanding gives the exact segment equation

$$2^{W+e-1}g + 2^W \delta 5^{j-1} = 5^d\, g + \sum_{i=0}^{d-1} 2^{W_i} 5^{d-1-i}.$$

This is the candidate form `orbitSegmentWord_candidate_equation`. No approximation enters: the equation is the word-orbit identity with the parameterisation substituted.

4.8 Size bounds used in the exclusions

The exclusion arguments use two size bounds on $g = \mathrm{Orbit}_7(j-1)$:

$$g < \frac{5^{j-1}}{4}, \qquad g < \frac{5^{j-1}}{2^{e-1}}.$$

The first is used by the $d = 1$ exclusion and by the $e \geq 3$ cases; the second, with $e \geq 2$, is used by the $d = 2$ size analysis. Both are consequences of the no-$H_{\geq}$ premises together with the exact orbit bound; in the Lean statements they are carried as hypotheses. The paper records them as premises rather than re-deriving them at each exclusion.

4.9 Segment exclusions

The exact segment equation is

$$2^{W+e-1}g + 2^W \delta 5^{j-1} = 5^d\, g + \sum_{i=0}^{d-1} 2^{W_i} 5^{d-1-i}.$$

For small d, the segment word molecule and total weight have the explicit forms:

d	$A(w)$	W
1	1	u_1 (in the $d = 1$ branch, $u_1 = 1 + 4a$)
2	$5 + 2^{u_1}$	$u_1 + u_2$
3	$25 + 5 \cdot 2^{u_1} + 2^{u_1+u_2}$	$u_1 + u_2 + u_3$

For the surviving branches the equations reduce as follows. With $\delta = 1$, $e = 2$, $u_1 = 1$ the $d = 2$ equation becomes

$$17g = 4 \cdot 5^{j-1} - 7.$$

With $u_1 = u_2 = 1$, $a = 0$ (so $u_3 = 1$) the $d = 3$ equation becomes

$$16g + 8 \cdot 5^{j-1} = 125g + 39,$$

hence

$$g = \frac{8 \cdot 5^{j-1} - 39}{109}.$$

The exclusions are:

1. $d = 1$: impossible by the size bound $g < 5^{j-1}/4$.
2. $d = 2$: modular contradiction modulo 16.
3. $d = 3$: first big step at depth $j + 4$ with weight 6.
4. $d \geq 4$: first big step weight at least 5, or the $e = 3, j = 17$ modular exclusion.

Status: all four exclusions are math-proven and Lean-compiled.

4.10 Worked $d = 1$ exclusion

The $d = 1$ segment equation is

$$5g + 1 = 2^{1+4a} \left(2^{e-1} g + \delta 5^{j-1}\right),$$

i.e.

$$5g + 1 = 2^{e+4a} g + \delta 2^{1+4a} 5^{j-1}.$$

If $e \geq 3$ or $a \geq 1$, then $2^{e+4a} \geq 8$, and the second term is positive, so the right-hand side exceeds $5g + 1$, contradicting the equality.

If $e = 2$ and $a = 0$, the equation reduces to

$$5g + 1 = 2\left(2g + \delta 5^{j-1}\right),$$

so

$$g + 1 = 2\delta 5^{j-1}.$$

Since $\delta \geq 1$, this gives $g \geq 2 \cdot 5^{j-1} - 1$, contradicting the size bound $g < 5^{j-1}/4$.

4.11 Full $d = 2$ size analysis

The $d = 2$ segment equation is

$$\left(2^{u_1+e} - 25\right) g + 2^{u_1+1} \delta 5^{j-1} = 5 + 2^{u_1}.$$

The right-hand side is at most 9 for $u_1 \in \{1, 2\}$, so the left-hand side must be small. In particular

$$2^{u_1+e} < 25,$$

which leaves only

$$(u_1, e) \in \{(1, 2), (1, 3), (2, 2)\}.$$

For $u_1 = 1, e = 2$, the equation becomes

$$17g = 4\delta 5^{j-1} - 7.$$

The size bound is $g < 5^{j-1}/2$. For $\delta = 3$ the right-hand side is about $12 \cdot 5^{j-1}/17$, which exceeds the bound; only $\delta = 1$ survives.

For $u_1 = 1, e = 3$, the equation becomes

$$9g = 8\delta 5^{j-1} - 7,$$

with the stronger size bound $g < 5^{j-1}/4$. Even $\delta = 1$ gives $g \approx 8 \cdot 5^{j-1}/9$, contradicting the bound.

For $u_1 = 2, e = 2$, the equation becomes

$$9g = 8\delta 5^{j-1} - 9,$$

with the size bound $g < 5^{j-1}/2$; again $\delta = 1$ already exceeds it.

Hence the only surviving branch is $\delta = 1$, $e = 2$, $u_1 = 1$. The subsequent modular contradiction is given in the next subsection.

4.12 Worked modular computation for $d = 2$

For $d = 2$ the surviving branch is $\delta = 1$, $e = 2$, $u_1 = 1$, which reduces the segment equation to

$$17g = 4 \cdot 5^{j-1} - 7.$$

Modulo 17 this gives

$$4 \cdot 5^{j-1} \equiv 7,$$

and since $4^{-1} \equiv 13 \pmod{17}$,

$$5^{j-1} \equiv 7 \cdot 13 \equiv 6 \pmod{17}.$$

The powers of 5 modulo 17 have order 16, and

$$5^3 = 125 \equiv 6 \pmod{17},$$

so $j - 1 \equiv 3 \pmod{16}$.

The candidate congruence $x \equiv 743 \pmod{1280}$ with $x = 2g + 5^{j-1}$ then forces

$$5^{j-1} \equiv 45 \pmod{256}.$$

Since $5^7 = 78125 \equiv 45 \pmod{256}$ and the order of 5 modulo 256 is 64, this gives $j-1 \equiv 7 \pmod{64}$. The two congruences contradict each other modulo 16.

4.13 Residue ledger

Case	Congruence and role	Lean
$d = 1$	no congruence; the size bound $g < 5^{j-1}/4$ suffices	`d1_exclusion`
$d = 2$	$5^{j-1} \equiv 6 \pmod{17}$ and $x \equiv 743 \pmod{1280}$ force $j - 1 \equiv 3 \pmod{16}$ and $5^{j-1} \equiv 45 \pmod{256}$	`d2_exclusion`
$d = 3$	$j - 1 \equiv 923 \pmod{1728}$ fixes the unique surviving family	`d3_family_big_weight_excluded`
$d \geq 4$	the $e = 3$, $j = 17$ branch is excluded by residues modulo 640 or 1280	`dge4_e3_j17_t1_excluded`, `dge4_e3_j17_t2_delta3_excluded`

4.14 Full $d = 3$ survivor derivation

The surviving $d = 3$ parameters are

$$t_j = 2, \qquad \delta = 1, \qquad e = 2, \qquad u_1 = u_2 = 1, \qquad a = 0,$$

which forces $u_3 = 1$. The segment word molecule is

$$A(w) = 25 + 5 \cdot 2^{u_1} + 2^{u_1+u_2} = 25 + 10 + 4 = 39,$$

and the total segment weight is

$$W = u_1 + u_2 + u_3 = 3.$$

The segment equation

$$2^{W+e-1}g + 2^W \delta 5^{j-1} = 5^d g + A(w)$$

therefore reads

$$2^{3+1}g + 2^3 \cdot 5^{j-1} = 5^3 g + 39,$$

i.e.

$$16g + 8 \cdot 5^{j-1} = 125g + 39,$$

so

$$109g = 8 \cdot 5^{j-1} - 39.$$

The integrality of g requires

$$8 \cdot 5^{j-1} \equiv 39 \pmod{109}.$$

The inverse of 8 modulo 109 is 41, and

$$39 \cdot 41 = 1599 \equiv 73 \pmod{109},$$

so

$$5^{j-1} \equiv 73 \pmod{109}.$$

The full family calculation combines this with the remaining branch congruences and produces the residue class

$$j - 1 \equiv 923 \pmod{1728}.$$

The consistency of the representative 923 is checked by

$$5^{923} \equiv 73 \pmod{109}, \qquad 8 \cdot 73 = 584 \equiv 39 \pmod{109}.$$

For this family the first step of weight at least 3 occurs at depth $j + 4$ and has weight 6. The finite base says that the first big step of the orbit of 7 is at depth 15 with weight exactly 3, so the family is excluded.

4.15 $d = 3$ family record

Item	Value
surviving parameters	$t_j = 2$, $\delta = 1$, $e = 2$, $u_1 = u_2 = 1$, $a = 0$
state formula	$g = (8 \cdot 5^{j-1} - 39)/109$
residue	$j - 1 \equiv 923 \pmod{1728}$
first big step	depth $j + 4$, weight 6
contradiction	finite base: first big step at depth 15 has weight 3

4.16 Full $d \geq 4$ branch analysis

For $d \geq 4$, the first big step of the candidate block is one of three types.

Branch	First big step	Contradiction
$e \geq 3$	$g_{\text{prev}} \to g$, the incoming step	finite base forces $e = 3$, $j = 17$; residue modulo 640 or 1280 fails
$e = 2$, $a \geq 1$	$y \to x$, weight $1 + 4a \geq 5$	the first big step of the prefix has weight exactly 3
$e = 2$, $a = 0$	$z \to w$, weight 5 or 6	the first big step of the prefix has weight exactly 3

In the $e \geq 3$ branch, the incoming step is already big. The finite base allows only $e = 3$ and $j = 17$, and in that case the candidate residue is excluded modulo 640 or 1280. The two compiled statements are `dge4_e3_j17_t1_excluded` and `dge4_e3_j17_t2_delta3_excluded`.

In the $e = 2$ branches, the first big step occurs inside the segment with weight at least 5. The finite base says the first big step of the orbit of 7 is unique, occurs at depth 15, and has weight exactly 3. A candidate first big step of weight 5 or 6 cannot occur at depth 15 and cannot occur before it; if it occurred later, the true depth-15 step would already be first. This is recorded by `candidate_first_big_step_weight_ge_five`.

4.17 $d \geq 4$ branch table

Branch	First big step	Contradiction
$e \geq 3$	$g_{\text{prev}} \to g$	finite base forces $e = 3$, $j = 17$; residue modulo 640 or 1280 fails
$e = 2$, $a \geq 1$	$y \to x$, weight $1 + 4a \geq 5$	the first big step of the prefix has weight exactly 3
$e = 2$, $a = 0$	$z \to w$, weight 5 or 6	the first big step of the prefix has weight exactly 3

For the $e = 2$ branches, the contradiction is the same finite fact: the first big step of the full orbit is unique, occurs at depth 15, and has weight exactly 3. A candidate first big step of weight 5 or 6 therefore cannot occur at depth 15 and cannot occur before it; if it occurred later, the depth-15 step would already be the first big step. The $e \geq 3$ branch is instead reduced by the finite base to $e = 3$, $j = 17$ and excluded by the stated residues.

The $d = 3$ residue class is consistent with the denominator in $g = (8 \cdot 5^{j-1} - 39)/109$: since

$$5^{923} \equiv 73 \pmod{109}, \qquad 8 \cdot 73 \equiv 39 \pmod{109},$$

the integer $8 \cdot 5^{923} - 39$ is divisible by 109. The derivation of the full modulus 1728 is part of the $d = 3$ family calculation.

4.18 CRT and terminal residue

Let $n = s - j$ and $\Delta = W_s - W_j$. The CRT candidate $r_{j,0}$ is the least nonnegative solution of

$$3 \cdot 5^n r_{j,0} + 3B + 2^\Delta = 2^{\Delta+L+4} \left(u + m' 2^{H_s - 1}\right),$$

with u the least nonnegative residue of $-5^{-(L+3)}$ modulo 2^{H_s-1}. The terminal residue is

$$y^* = -\left(w_{\text{term}} + 5^{-(L+3)}\right)(3 \cdot 5^s)^{-1} \pmod{2^{H_s-1}},$$

and the final inequality is equivalent to $y^* \neq 0$.

The Lean names are `rj0_spec_eq`, `terminal_bound_iff_yStar_ne_zero`, and `rj0_ge_of_size_conditions_no_hge`.

4.19 Finite base

The first 17 states are explicitly expanded:

$$7, 9, 23, 29, 73, 183, 229, 573, 1433, 3583, 4479, 5599, 6999, 8749, 21873, 54683, 34177.$$

The first big step is at depth 15 with weight 3.

Status: math-proven and Lean-compiled in `FinitePrefix.lean`, with no `native_decide`.

4.20 Finite-prefix facts used by the exclusions

The exclusions use the following finite facts:

- the first 17 states are expanded exactly;
- the step weights for $n < 16$ are expanded exactly;
- the first step of weight at least 3 is at depth 15 and has weight exactly 3;
- the first big step is unique inside this prefix;
- a candidate first big step of weight 6 contradicts the exact weight 3 at depth 15;
- a candidate first big step of weight at least 5 cannot be the first big step of the prefix.

The corresponding Lean names are listed in the Lean theorem index: `fullOrbitIter_prefix_expand`, `fullOrbit_prefix_step_weights`, `fullOrbit_first_t_ge3_is_exactly_3`, `first_big_step_unique`, `candidate_first_big_step_weight_ne_three`, and `candidate_first_big_step_weight_ge_five`.

4.21 Final valuation inequality

Theorem 4.5. *Under the no-$H_{\geq}$ premises,* FullOrbitFrom7(r), *and* $2 \leq H_s$,

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \leq H_s - 2,$$

where $w_{\mathrm{term}} = (3r_s + 1)/2^{L+4}$.

Status: math-proven; Lean pending in `UnifiedCoreAudit.unified_core_final_no_hge`.

5 Divergence bridge

This chapter expands the cycle bridge and the DWDB-DIV assembly.

5.1 Cycle words

A positive cycle gives a closed word

$$\mathrm{wordOrbit}(w, \mathrm{Orbit}_7(c)) = \mathrm{Orbit}_7(c).$$

The word is split into rise steps ($t = 1$ or $t = 2$) and C3 steps ($t \geq 3$).

5.2 Construction of the period word

Define the period word recursively by

$$\mathrm{cycleWord}(c, 0) = [],$$

$$\mathrm{cycleWord}(c, p+1) = \mathrm{cycleWord}(c, p) +\!+ [v_2(5\,\mathrm{Orbit}_7(c+p) + 1)]\,.$$

By induction on p,

$$\mathrm{wordOrbit}(\mathrm{cycleWord}(c, p), \mathrm{Orbit}_7(c)) = \mathrm{Orbit}_7(c+p),$$

and each appended entry is exact by definition. Hence the compiled lemmas `cycleWord_step_exact` and `cycleWord_take_eq` hold. If $\mathrm{Orbit}_7(c) = \mathrm{Orbit}_7(c+p)$, the word is closed and `cycleWord_cycle_equation` applies.

5.3 Cycle word lemmas

The following Lean lemmas extract the cycle word and its QB-8 split:

- `orbit_cycle_imp_min_rise_start`;
- `orbit_reset_head_eq_of_rise_start`;
- `orbit_cycle_imp_min_rise_witness`;
- `cycleWord_step_exact`;
- `cycleWord_take_eq`;
- `cycleQb8Input_exists_rise_index`;
- `cycleQb8Input_exists_c3_index`;
- `cycleQb8Input_exists_block_head_mod_five`;
- `rise_block_balance`.

5.4 Extraction and split proof sketch

Given a positive cycle, choose equal orbit values at depths $m < n$ and put $c = m$, $p = n - m$, $m_0 = \mathrm{Orbit}_7(c)$. Define the word by

$$t_i = v_2(5\,\mathrm{Orbit}_7(c+i)+1)\,, \qquad 0 \le i < p.$$

The compiled lemmas `cycleWord_step_exact`, `cycleWord_take_eq`, and `cycleWord_cycle_equation` give validity, closedness, and the exact cycle equation.

The QB-8 split is obtained by filtering the word into rise entries $t < 3$ and C3 entries $t \ge 3$. Closedness implies both filters are nonempty: a cycle cannot consist only of rise steps, because their total weight would be at most $2P$, contradicting the cycle equation $2^W m = 5^P m + A(w)$; and it cannot consist only of C3 steps, because each such step satisfies $(5x+1)/2^t < x$ for $x \ge 1$, so the states would strictly decrease and could not close. The precise statements are `cycleQb8Input` and `cycleQb8Input_P_ge_two`.

5.5 The QB-8 input record

The Lean structure `CycleQb8Input m S P w rise c3` records everything that follows from a positive cycle without range assumptions:

- `hvalid`: the word w is valid from m;
- `hclosed`: $\mathrm{wordOrbit}(w, m) = m$;
- `hlength`, `hweight`: length P and total weight S;
- `hrise_ok`, `hc3_ok`: rise entries lie in $\{1, 2\}$ and C3 entries are at least 3;
- `hlen_split`, `hweight_split`: the rise and C3 lists decompose the word and its weight;
- `hrise_filter`, `hc3_filter`: the split is the exact filter split;

- `hc3_pos`, `hrise_pos`: both parts are nonempty;
- `hexact`: every word step equals the orbit valuation;
- `hm_pos`, `hm_odd`, `hm_not_five`: m is positive, odd, and not divisible by 5.

The structure is produced by `orbit_cycle_imp_cycle_qb8_input`.

5.6 Failure window fields

The structure `FailureWindow m S P w rise c3 j k0 t delta s` adds to the QB-8 input:

- `hj_lt`: $j < P$;
- `ht`: $t \in \{1, 2\}$;
- `hdelta`: $\delta = 1$ when $t = 1$, and $\delta \in \{1, 3\}$ when $t = 2$;
- `hreset`: the exact reset equation with the prefix state $\mathrm{wordOrbit}(w\lceil j, m)$;
- `hreach`: the corrected reachability predicate $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$;
- `hs_odd`, `hs_not_five`, `hk`, `hs_lt`: parity, coprimality, and size conditions;
- `hfail_t1`, `hfail_t2`: the valuation lower bounds $2j + 12$ and $2j + 11$.

5.7 Construction checklist

The intended construction from a closed QB-8 word is conditional on the unified core; the statement `failureWindowExistence` is not claimed as proved in Lean.

1. Extract the closed word and its rise/C3 split.
2. Choose the block head from the rise and C3 position lemmas.
3. Obtain the reset equation and the corrected reachability witness.
4. Apply the unified core to the block tail.
5. Read off the valuation lower bound.
6. Compare with the corrected decisive window bound.

5.8 Corrected window bound

The corrected decisive window bound requires $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$ and asserts

$$v_2\left(5^{k_0+1}s + \delta 5^j\right) \le 2j + 10 \qquad (t = 2),$$

$$v_2\left(5^{k_0+1}s + 5^j - 2\right) \le 2j + 11 \qquad (t = 1).$$

The old unconstrained bound is false; the counterexample is

$$j = 36,\ k_0 = 0,\ t = 2,\ \delta = 3,\ s = 2^{83} - 3 \cdot 5^{35},$$

with valuation $83 > 82$. The corrected predicate ResetWindowReachability(j, k_0, t, δ, s) fixes the exact block parameters and carries full orbit reachability.

The valuation is 83 because the 5-adic parts cancel exactly:

$$5s + 3 \cdot 5^{36} = 5(2^{83} - 3 \cdot 5^{35}) + 3 \cdot 5^{36} = 5 \cdot 2^{83}.$$

The witness satisfies the size, parity, and coprimality conditions; it fails only because it carries no orbit reachability data.

5.9 Corrected bound as a projection

The corrected bound is not introduced as a separate analytic hypothesis. In the Lean repository it is recorded as a definition with the unified core as an input:

- `unifiedCoreClosed`: the unified-core valuation inequality under the no-$H_{\geq}$ premises, reachability, and $2 \leq H_s$;
- `decisiveWindowValuationBoundCorrected_of_unified_core`: `unifiedCoreClosed` → `BlockAutomaton.decisiveWindowValuationBoundCorrected`;
- `failureWindowExistenceOfUnifiedCore`: `unifiedCoreClosed` → `CycleBridge.failureWindowExistence`.

These are conditional interfaces. None of them claims that the unified core is already closed in Lean.

5.10 Projection field mapping

`ResetWindowReachability` field	Role	Unified-core input
$s5^{k_0} = r + 1$	previous terminal	block-tail terminal
GeneralOrbitFrom7(r)	terminal reachability	general-orbit input
FullOrbitFrom7(r_j)	reset-head reachability	block-head reachability
ResetHeadEq	exact reset equation	block equation
$s < 5^{j-1-k_0}$, oddness, $5 \nmid s$	size and parity	candidate size

5.11 Corrected bound statement shape

```
def t2WindowBoundCorrected (j k0 a t delta s : Nat) : Prop :=
  a = j - k0 - 1 ->
  t = 2 -> delta = 1 \/ delta = 3 ->
  S6Audit.ResetWindowReachability j k0 t delta s ->
  t2WindowValue j k0 delta s <= 2 * j + 10

def t1WindowBoundCorrected (j k0 a t s : Nat) : Prop :=
  a = j - k0 - 1 -> t = 1 ->
  S6Audit.ResetWindowReachability j k0 t 1 s ->
  t1WindowValue j k0 s <= 2 * j + 11

def decisiveWindowValuationBoundCorrected : Prop :=
  (forall j k0 a t delta s,
     t2WindowBoundCorrected j k0 a t delta s) /\
```

```
(forall j k0 a t s,
   t1WindowBoundCorrected j k0 a t s)
```

5.12 No cycle implies unbounded: full proof

Assume that the orbit of x is bounded by B. Then every value lies in the finite set $\{0, \ldots, B\}$, so the pigeonhole principle gives indices $m < n$ with $\mathrm{Orbit}_x(m) = \mathrm{Orbit}_x(n)$. This is a positive cycle. Contrapositively, the absence of a positive cycle implies that the orbit is unbounded. The statement is `unbounded_of_no_cycle`.

5.13 Failure windows

A failure window is a reachable reset block whose valuation lower bound exceeds the corrected upper bound. Its existence is `failureWindowExistence`, an input to the assembly.

5.14 Assembly

$$\text{corrected window bound} \;\Rightarrow\; \neg\mathrm{OrbitCycle}(7) \;\Rightarrow\; \mathrm{IsUnboundedOrbit}(7).$$

The Lean interfaces are:

- `CycleBridge.windowBoundToNoCycle`;
- `CycleBridge.failure_window_contradicts_window_corrected`;
- `DwdbDiv.dwdbDivFinalAssembly`.

All downstream statements that consume the corrected window bound or the failure-window existence are conditional interfaces; they do not claim those inputs are already proved.

5.15 Failure window contradiction

For $t = 2$, the failure window gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \geq 2j + 11,$$

while the corrected bound gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \leq 2j + 10.$$

For $t = 1$, the lower bound is $2j + 12$ and the upper bound is $2j + 11$. Both are contradictions. The Lean theorem is `failure_window_contradicts_window_corrected`.

5.16 Contradiction lemma shapes

The two elementary contradiction lemmas are:

```
theorem failure_window_contradicts_t2
    (j k0 delta s v : Nat)
    (hfail : 2 * j + 11 <= v)
    (hwin : v <= 2 * j + 10) : False
```

```
theorem failure_window_contradicts_t1
    (j k0 s v : Nat)
    (hfail : 2 * j + 12 <= v)
    (hwin : v <= 2 * j + 11) : False
```

5.17 DwdbDiv assembly

The final assembly theorems are:

- `DwdbDiv.dwdbDivFinalAssembly`;
- `DwdbDiv.five_x_plus_one_diverges_at_7_of_window_bound`;
- `DwdbDiv.cycle_of_window_bound_contradiction`;
- `DwdbDiv.five_x_plus_one_diverges_at_7_of_failure_window`.

These theorems are conditional on the corrected window bound and on `failureWindowExistence`; neither input is claimed as already proved.

6 The arithmetic counterexample

This chapter dissects the counterexample that forces the corrected decisive window bound to carry orbit reachability.

6.1 The witness

The unconstrained window bound claims

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \le 2j + 10$$

for every choice of j, k_0, t, δ, s satisfying the arithmetic conditions. The witness

$$j = 36, \quad k_0 = 0, \quad t = 2, \quad \delta = 3, \quad s = 2^{83} - 3 \cdot 5^{35}$$

has valuation

$$v_2\big(5s + 3 \cdot 5^{36}\big) = v_2\big(5 \cdot 2^{83}\big) = 83,$$

which is one more than the claimed bound

$$2j + 10 = 82.$$

The cancellation is exact:

$$5s + 3 \cdot 5^{36} = 5\left(2^{83} - 3 \cdot 5^{35}\right) + 3 \cdot 5^{36} = 5 \cdot 2^{83}.$$

6.2 Exact decimal values

The comparison underlying the size condition is

$$2^{81} = 2417851639229258349412352 < 291038304567337036132812 5 = 5^{35}.$$

The witness itself is

$$s = 2^{83} - 3 \cdot 5^{35} = 940257419896922313665033.$$

All three decimal values are exact; no rounding enters the proof.

6.3 Verification of the arithmetic hypotheses

The witness satisfies every arithmetic condition that the old bound used.

- **Size.** From $2^{81} < 5^{35}$,
$$s = 2^{83} - 3 \cdot 5^{35} < 4 \cdot 5^{35} - 3 \cdot 5^{35} = 5^{35},$$
which is the required bound $s < 5^{j-k_0-1} = 5^{35}$.
- **Oddness.** 2^{83} is even and $3 \cdot 5^{35}$ is odd, so s is odd.
- **Coprimality with 5.** Since $2^4 \equiv 1 \pmod 5$ and $83 \equiv 3 \pmod 4$,
$$2^{83} \equiv 2^3 \equiv 3 \pmod 5,$$
and $3 \cdot 5^{35} \equiv 0 \pmod 5$, so $s \equiv 3 \pmod 5$ and $5 \nmid s$.
- **The valuation.** The cancellation identity gives $v_2\big(5s + 3 \cdot 5^{36}\big) = 83$, as shown above.

6.4 What the witness does not carry

The witness is arithmetic-only. It supplies no previous terminal state r, no general-orbit reachability GeneralOrbitFrom7(r), no reset head r_j, and no full-orbit reachability FullOrbitFrom7(r_j). In particular there is no exact reset equation

$$\text{ResetHeadEq}(s, 36, 0, 2, 3, r_j)$$

with a full-orbit r_j attached to the witness. The old bound failed because it treated the arithmetic conditions as sufficient.

6.5 The corrected predicate

The corrected decisive window bound requires

$$\text{ResetWindowReachability}(j, k_0, t, \delta, s),$$

which fixes the same j, k_0, t in every field:

- the previous-terminal equation $s5^{k_0} = r + 1$;
- GeneralOrbitFrom7(r);
- FullOrbitFrom7(r_j);
- the exact reset equation ResetHeadEq($s, j, k_0, t, \delta, r_j$);
- the size bound, oddness, and $5 \nmid s$.

The arithmetic witness is excluded by the reachability fields, not by any change in the valuation threshold. The constants $2j + 10$, $2j + 11$, and $2j + 12$ are unchanged.

6.6 How the corrected bound is obtained

The corrected bound is a projection of the unified core. In the Lean repository it is defined as an interface:

```
def t2WindowBoundCorrected (j k0 a t delta s : Nat) : Prop :=
  a = j - k0 - 1 ->
  t = 2 -> delta = 1 \/ delta = 3 ->
  S6Audit.ResetWindowReachability j k0 t delta s ->
  t2WindowValue j k0 delta s <= 2 * j + 10
```

The implication is conditional on `ResetWindowReachability`; it is not a standalone arithmetic statement.

6.7 The failure-window role

The counterexample is also the reason why `failureWindowExistence` must be proved from the closed cycle word rather than from size estimates alone. A positive cycle would produce a witness with the reachability fields automatically; the arithmetic-only witness shows that the fields are essential. The construction of the reachable failure window is the pending input of the divergence bridge.

6.8 Status

Statement	Status
arithmetic hypotheses of the witness	verified exactly
corrected window bound definition	Lean: compiled
`ResetWindowReachability` definition	Lean: compiled
failure-window construction from a cycle	Lean: pending

7 Failure window construction

This chapter records the intended construction of a failure window from a positive cycle. The construction is the mathematical content of `failureWindowExistence`; in the Lean repository that statement remains pending, while every named ingredient below is either compiled or compiled as a conditional interface.

7.1 Input: a positive cycle

Assume

$$\text{OrbitCycle}(7).$$

The compiled cycle-word layer produces a starting depth c, a period p, a state $m = \text{Orbit}_7(c)$, and a word w of length p such that

- w is valid from m;
- $\text{wordOrbit}(w, m) = m$;
- every entry of w is the exact step weight at the corresponding orbit position;

- the cycle equation
$$2^{W(w)}\, m = 5^p\, m + A(w)$$
holds.

This is the compiled input `orbit_cycle_imp_cycle_qb8_input`.

7.2 The QB-8 split

The word is filtered into rise entries and C3 entries:

$$\text{rise} = \{t \in w : t < 3\}, \qquad \text{c3} = \{t \in w : t \geq 3\}.$$

Both lists are nonempty:

- a cycle cannot consist only of rise steps, because their total weight is at most $2P$, contradicting the cycle equation;
- a cycle cannot consist only of C3 steps, because every C3 step satisfies $(5x+1)/2^t < x$ for $x \geq 1$, so the states would strictly decrease and could not close.

The compiled lemmas `cycleQb8Input_exists_rise_index` and `cycleQb8Input_exists_c3_index` provide genuine positions inside the word.

7.3 The reset start

The construction chooses a global-minimum rise start. The compiled extraction lemmas

- `orbit_cycle_imp_min_rise_start`;
- `orbit_reset_head_eq_of_rise_start`;
- `orbit_cycle_imp_min_rise_witness`;

produce parameters j, k_0, t, δ, s and states r, r_j such that

- $t \in \{1, 2\}$ and $\delta = 1$ for $t = 1$, $\delta \in \{1, 3\}$ for $t = 2$;
- $s5^{k_0} = r + 1$;
- GeneralOrbitFrom7(r) and FullOrbitFrom7(r_j);
- the reset equation ResetHeadEq$(s, j, k_0, t, \delta, r_j)$ holds;
- $s < 5^{j-1-k_0}$, s is odd, and $5 \nmid s$.

This is exactly the corrected reachability predicate ResetWindowReachability(j, k_0, t, δ, s).

7.4 The block decomposition

The block head r_j is followed by a tail of length L ending at the terminal state r_s. Let

$$n = s - j, \qquad \Delta = W_s - W_j,$$

and let B be the block molecule of the tail. The tail is a valid word, so the exact word-orbit identity gives

$$2^{\Delta}\, r_s = 5^n\, r_{j,0} + B.$$

The unified core studies exactly this block: the CRT candidate, the terminal residue, and the size conditions live in the coordinates of this tail.

7.5 The unified-core projection

The corrected decisive window bound is a projection of the unified core. In the Lean repository the interface

```
decisiveWindowValuationBoundCorrected_of_unified_core :
  unifiedCoreClosed -> decisiveWindowValuationBoundCorrected
```

is compiled as a conditional interface. Its input `unifiedCoreClosed` is the mathematical statement whose Lean representative is `UnifiedCoreAudit.unified_core_final_no_hge`; that statement is pending.

7.6 The block-layer lower bound

The compiled lemma `rise_block_balance` gives the valuation lower bound from the reset block:

$$t = 2: \qquad v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \geq 2j + 11,$$

$$t = 1: \qquad v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \geq 2j + 12.$$

The lower bound comes from the exact block-layer count of the rise and C3 steps, not from a size estimate.

7.7 The corrected upper bound

For a reachable reset window, the corrected decisive window bound gives

$$t = 2: \qquad v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \leq 2j + 10,$$

$$t = 1: \qquad v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \leq 2j + 11.$$

The two bounds contradict the lower bounds above. The contradiction is packaged by the compiled lemma `failure_window_contradicts_window_corrected`.

7.8 The failure window record

The construction assembles the fields of

$$\mathrm{FailureWindow}(m, S, P, w, \mathrm{rise}, \mathrm{c3}, j, k_0, t, \delta, s):$$

- `hj_lt`: $j < P$;
- `ht`: $t \in \{1, 2\}$;
- `hdelta`: the matching δ;
- `hreset`: the exact reset equation with the prefix state $\mathrm{wordOrbit}(w\lceil j, m)$;
- `hreach`: $\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s)$;
- `hs_odd`, `hs_not_five`, `hk`, `hs_lt`: parity, coprimality, and size;
- `hfail_t1`, `hfail_t2`: the valuation lower bounds.

The existence of this record is `failureWindowExistence`, pending.

7.9 The full chain

```
OrbitCycle 7
  -> cycleQb8Input (compiled)
  -> rise and C3 indices (compiled)
  -> reset start witness (compiled)
  -> reset equation and reachability (compiled)
  -> block tail and unified core (unifiedCoreClosed pending)
  -> decisive window bound (compiled conditional)
  -> rise_block_balance lower bound (compiled)
  -> FailureWindow fields (assembly pending)
  -> failure_window_contradicts_window_corrected (compiled)
  -> not OrbitCycle 7 (compiled conditional)
  -> IsUnboundedOrbit 7 (compiled conditional)
```

7.10 What is and is not claimed

Every step before the failure-window assembly is either compiled or a direct projection of the unified core. The paper does not claim that `failureWindowExistence` is compiled: it is the pending input of the cycle bridge. The construction also does not claim that the unified core is closed; that closure is exactly the pending statement `unified_core_final_no_hge`.

7.11 Status ledger

Statement	Status
cycle-word extraction and QB-8 split	compiled
rise-start reset witness	compiled
block-layer lower bound	compiled
failure-window contradiction lemma	compiled
corrected bound as a projection	compiled (conditional)
`failureWindowExistence`	pending
`unified_core_final_no_hge`	pending

8 Worked computations ledger

This chapter collects the exact computations used in the paper into a single reference ledger. Every entry is an exact integer identity. The formal proof does not rely on this ledger as a decision procedure: the corresponding facts are proved in Lean by explicit rewriting.

8.1 Orbit and prefix ledger

n	$\mathrm{Orbit}_7(n)$	t_n	W_n	A_n	$5^n \cdot 7 + A_n$	$2^{W_n}\mathrm{Orbit}_7(n)$
0	7	–	0	0	7	7
1	9	2	2	1	36	36
2	23	1	3	9	184	184
3	29	2	5	53	928	928
4	73	1	6	297	4672	4672
5	183	1	7	1549	23424	23424
6	229	2	9	7873	117248	117248
7	573	1	10	39877	586752	586752
8	1433	1	11	200409	2934784	2934784
9	3583	1	12	1004093	14675968	14675968
10	4479	2	14	5024561	73383936	73383936
11	5599	2	16	25139189	366936064	366936064
12	6999	2	18	125761481	1834745856	1834745856
13	8749	2	20	629069549	9173991424	9173991424
14	21873	1	21	3146396321	45871005696	45871005696
15	54683	1	22	15734078757	229357125632	229357125632
16	34177	3	25	78674588089	1146789822464	1146789822464

The last two columns are equal by the exact word-orbit identity

$$2^{W_n}\,\mathrm{Orbit}_7(n) = 5^n \cdot 7 + A_n.$$

For example, the final row reads

$$2^{25} \cdot 34177 = 5^{16} \cdot 7 + 78674588089 = 1146789822464.$$

8.2 Block equations

$$\begin{aligned} 2^3 \cdot 23 &= 184 = 5^2 \cdot 7 + 9 && \text{block } [2,1] \text{ from } 7,\\ 2^3 \cdot 29 &= 232 = 5^2 \cdot 9 + 7 && \text{block } [1,2] \text{ from } 9,\\ 2^4 \cdot 229 &= 3664 = 5^3 \cdot 29 + 39 && \text{block } [1,1,2] \text{ from } 29. \end{aligned}$$

The molecules are $A([2,1]) = 9$, $A([1,2]) = 7$, and $A([1,1,2]) = 39$.

8.3 PMI sums

For the exact prefix word $[2,1,2]$ from 7,

$$\sum_{j=0}^{2} 2^{-m_j} = 1 + \frac{4}{5} + \frac{8}{25} = \frac{53}{25} = \frac{5A(w)}{5^3}.$$

For the word $[3,1]$, the cleared sum is

$$\mathrm{aTotal5}(2,t) = 5^2 + 2^3 \cdot 5^1 = 25 + 40 = 65 = 5A([3,1]).$$

The first example has no bad prefix; the second has exactly one.

8.4 Reduced segment equations

$$\begin{aligned} d=1: &\qquad 5g+1 = 2^{1+4a}\left(2^{e-1}g+\delta 5^{j-1}\right),\\ d=2: &\qquad \left(2^{u_1+e}-25\right)g + 2^{u_1+1}\delta 5^{j-1} = 5+2^{u_1},\\ d=3,\ \text{survivor}: &\qquad 16g + 8\cdot 5^{j-1} = 125g+39. \end{aligned}$$

The $d=3$ survivor therefore satisfies

$$g = \frac{8\cdot 5^{j-1}-39}{109}.$$

8.5 Modular tables

Modulo 17, the powers of 5 satisfy

$$5^1 \equiv 5, \quad 5^2 \equiv 8, \quad 5^3 \equiv 6 \pmod{17},$$

and the order is 16. Modulo 256,

$$5^7 = 78125 \equiv 45 \pmod{256},$$

and the order is 64. Modulo 109, repeated squaring gives:

k	$5^k \bmod 109$	k	$5^k \bmod 109$	k	$5^k \bmod 109$
1	5	16	89	256	26
2	25	32	73	512	22
4	80	64	97	923	73
8	78	128	35		

The entry $5^{923} \equiv 73$ is the product

$$22\cdot 26\cdot 35\cdot 89\cdot 78\cdot 25\cdot 5 \equiv 73 \pmod{109},$$

and $8\cdot 73 = 584 \equiv 39 \pmod{109}$, so the $d=3$ denominator is consistent.

8.6 Arithmetic counterexample

The witness is

$$s = 2^{83} - 3\cdot 5^{35} = 9402574198969223136650 33.$$

The size comparison is

$$2^{81} = 2417851639229258349412352 < 291038304567337036132 8125 = 5^{35},$$

and the valuation identity is

$$5s + 3\cdot 5^{36} = 5\cdot 2^{83} = 48357032784585166988247040.$$

Hence $v_2\left(5s+3\cdot 5^{36}\right) = 83$, while the unconstrained bound is 82. The witness is odd, satisfies $s \equiv 3 \pmod 5$, and fails only the orbit-reachability fields.

8.7 Verification note

Every entry in this ledger is an exact integer identity. The paper uses these numbers only to illustrate the algebra; the formal proof re-derives the corresponding facts by explicit rewriting in `FinitePrefix.lean`, `WordWindow.lean`, and `Pmi.lean`. No entry here is a substitute for a Lean theorem.

9 Theorem-by-theorem status matrix

This chapter records the status of every theorem consumed by the main chain. The matrix is organised by file. The statuses are the current audit state; they are re-checked whenever the repository changes.

9.1 Legend

- **Math: proven**: the mathematical argument is written out in this paper or the manual.
- **Lean: compiled**: the Lean theorem compiles without `sorry`.
- **Lean: pending**: the Lean theorem is not yet closed.
- **compiled (conditional)**: the theorem compiles but consumes an open input such as `unifiedCoreClosed` or `failureWindowExistence`.

9.2 WordWindow.lean

Statement	Role	Math	Lean
`wordA`	word molecule	proven	compiled
`wordOrbit`	word orbit	proven	compiled
`wordValid`	validity predicate	proven	compiled
word-orbit identity	exact ledger	proven	compiled
valuation toolkit	2-adic facts	proven	compiled

9.3 Pmi.lean

Statement	Role	Math	Lean
`aTotal5` clearing	PMI identity	proven	compiled
`pmi_algebraic`	cycle form	proven	compiled
`pmi_b_count`	counting bound	proven	compiled
`pmi_b_no_bad_prefix`	small budget	proven	compiled
margin balance	weight counts	proven	compiled
margin balance strict	closed words	proven	compiled

9.4 FinitePrefix.lean

Statement	Role	Math	Lean
`fullOrbitIter_prefix_expand`	seventeen states	proven	compiled
`fullOrbit_prefix_step_weights`	exact weights	proven	compiled
`fullOrbit_first_t_ge3_ is_exactly_3`	first big step	proven	compiled
`first_big_step_unique`	uniqueness	proven	compiled
`candidate_first_big_step_weight_ne_three`	d=3 use	proven	compiled
`candidate_first_big_step_weight_ge_five`	$d \geq 4$ use	proven	compiled

9.5 UnifiedCoreBridge.lean

Statement	Role	Math	Lean
`reset_head_predecessor`	reset predecessor	proven	compiled
`candidateX_of_reset_and_terminal`	candidate X	proven	compiled
`first_block_terminal_eq`	terminal state	proven	compiled
`reset_q0_form`	q0 interval	proven	compiled
`block_head_identity_of_reset`	block identity	proven	compiled
`q0_interval_iff_rj_bound`	interval form	proven	compiled
`orbitSegmentWord_equation`	segment identity	proven	compiled
`orbitSegmentWord_candidate_equation`	candidate form	proven	compiled
`d1_exclusion`	d=1	proven	compiled
`d2_exclusion`	d=2	proven	compiled
`d3_family_big_weight_excluded`	d=3	proven	compiled
`dge4_e2_a_ge1_excluded`	$d \geq 4$ branch	proven	compiled
`dge4_e3_j17_t1_excluded`	e=3 branch	proven	compiled
`dge4_e3_j17_t2_delta3_excluded`	e=3 branch	proven	compiled
full derivation from premises and FullOrbitFrom7	candidate layer	proven	pending

9.6 UnifiedCoreAudit.lean

Statement	Role	Math	Lean
`All36_20PremisesNoHge`	premises record	definition	compiled
`blockB_bound_of_no_hge`	tail bound	proven	compiled
`rj0_spec_eq`	CRT equation	proven	compiled
`terminal_bound_iff_yStar_ne_zero`	residue equivalence	proven	compiled
`rj0_ge_of_size_conditions_no_hge`	lower bound	proven	compiled
`unified_core_final_no_hge`	final inequality	proven	pending

9.7 CycleBridge.lean

Statement	Role	Math	Lean
`orbit_cycle_imp_min_rise_start`	rise start	proven	compiled
`orbit_reset_head_eq_of_rise_start`	reset equation	proven	compiled
`orbit_cycle_imp_min_rise_witness`	witness data	proven	compiled
`orbit_cycle_imp_cycle_qb8_input`	closed word	proven	compiled
`cycleWord_step_exact`	exact weights	proven	compiled
`cycleWord_take_eq`	prefixes	proven	compiled
`cycleWord_cycle_equation`	cycle equation	proven	compiled
`CycleQb8Input`	split record	definition	compiled
`cycleQb8Input_exists_rise_index`	rise position	proven	compiled
`cycleQb8Input_exists_c3_index`	C3 position	proven	compiled
`rise_block_balance`	lower bound	proven	compiled
`failure_window_contradicts_window_corrected`	contradiction	proven	compiled
`failureWindowExistence`	window existence	proven	pending
`windowBoundToNoCycle_of_failureWindowExistence`	bridge	proven	compiled (conditional)
`unbounded_of_no_cycle`	final step	proven	compiled

9.8 BlockAutomaton.lean

Statement	Role	Math	Lean
`t1WindowBoundCorrected`	t=1 bound	proven	compiled
`t2WindowBoundCorrected`	t=2 bound	proven	compiled
`decisiveWindowValuationBoundCorrected`	corrected bound	proven	compiled
`decisiveWindowValuationBoundCorrected_of_unified_core`	projection	proven	compiled (conditional)
`failureWindowExistenceOfUnifiedCore`	projection	proven	compiled (conditional)

9.9 DwdbDiv.lean and FinalStatement.lean

Statement	Role	Math	Lean
`dwdbDivFinalAssembly`	final assembly	proven	compiled (conditional)
`five_x_plus_one_diverges_at_7_of_window_bound`	assembly variant	proven	compiled (conditional)
`five_x_plus_one_diverges_at_7`	main theorem	proven	pending

9.10 How to update the matrix

The matrix is updated only after the following checks:

1. `lake build` passes for the affected files;
2. `#print axioms` returns only the allowlist for the affected theorems;
3. the `sorry` search is re-run for `UnifiedCoreAudit.lean`;
4. the `native_decide` search is re-run for the main chain.

No row is promoted from pending to compiled on the basis of this document alone.

10 Genericity ledger for (a, b)

The string-flow framework is written for the accelerated map

$$T_{a,b}(x) = \frac{ax + b}{2^{v_2(ax+b)}}.$$

This chapter records which parts of the framework are generic in a, which parts are specific to $(a, b) = (5, 1)$, and which parts are not claimed. The ledger does not prove any new statement for other parameters.

10.1 Status labels

- **Generic**: the statement holds for the indicated family of parameters with the same proof shape.
- **Parameter-dependent**: the statement shape is generic, but its constants or hypotheses depend on a and must be re-derived.
- $(5, 1)$**-specific**: the statement is proved only for $(a, b) = (5, 1)$.
- **Not claimed**: the statement is not asserted in this paper for other parameters.

10.2 The ledger

Layer	Status for (a, b)	Remark
words, validity, weight, molecule	Generic	replace 5 by a in the recursion and in the explicit sum
word-orbit identity	Generic	$2^{W(w)}x_L = a^L x_0 + A(w)$
PMI identity	Generic	$\alpha = \log_2 a$ replaces $\log_2 5$
cycle equation	Generic	$2^T m = a^P m + A(w)$ for a closed word
PMI-B counting bound	Generic	the same bad-prefix count with a^P
margin balance	Generic as an inequality	the weight classes $1, 2, \geq 3$ and the constants $\alpha - 1, \alpha - 2, 3 - \alpha$ depend on a
rise-step residue rules	Parameter-dependent	which weights count as rise steps depends on a
reset equation form	Parameter-dependent	the form generalises; δ and t data are $(5, 1)$-specific
finite prefix of 7	$(5, 1)$-specific	the exact 17-state expansion uses the orbit of 7
segment-size constants	$(5, 1)$-specific	constants such as 4 in $g < 5^{j-1}/4$ come from $a = 5$
modular exclusions for $d = 2, d = 3$	$(5, 1)$-specific	moduli $17, 256, 109$ and residues are arithmetic of 5
CRT candidate and terminal residue	$(5, 1)$-specific	powers of 5 and $3 \cdot 5^s$ enter the definitions
corrected window thresholds	$(5, 1)$-specific	$+10, +11, +12$ and the constant 13 are not transferred
counterexample arithmetic	$(5, 1)$-specific	$j = 36$, $s = 2^{83} - 3 \cdot 5^{35}$ is specific to $a = 5$
$a = 3$ analogue of the unified core	Not claimed	requires a new proof
general divergence behaviour	Not claimed	the paper proves only the orbit of 7 for $(5, 1)$

10.3 What the generic layer gives

The generic layer is the exact word ledger:

$$2^{W(w)}\, x_L = a^L x_0 + A(w),$$

the PMI identity

$$\sum_{j=0}^{L-1} 2^{-m_j} = \frac{a\, A(w)}{a^L}, \qquad m_j = j \log_2 a - W_j,$$

and the cycle equation. These identities are independent of the starting value and of the arithmetic of a beyond the word being valid. They are the part of the framework that can be reused without new proof.

10.4 What is not transferred

Everything after the word ledger is tied to the concrete arithmetic: the finite prefix, the segment exclusions, the modular exclusions, the CRT residue, and the decisive window thresholds. In particular, the constant 13 and the thresholds +10, +11, +12 are proved for $(5, 1)$; this paper does not claim they survive for $a = 3$.

10.5 The first obstruction question

The paper does not assert which lemma would be the first one that fails for $a = 3$. Within the $(5, 1)$ proof, the earliest layer where the parameter enters arithmetically is the segment-size layer ($g < 5^{j-1}/4$ in the $d = 1$ exclusion), followed by the rise-step residue rules and the decisive-window thresholds. A concrete answer for $a = 3$ would require re-developing the segment exclusions and the window arithmetic; that development is not part of this paper and is therefore marked as not claimed.

10.6 Status

Item	Status
generic word ledger	established for the framework
PMI and cycle identities	established for the framework
$(5, 1)$-specific core and bridge	proved in this paper
$a = 3$ analogue	not claimed
general divergence behaviour	not claimed

11 Cycle word algebra

This chapter develops the algebra of the cycle word: the recursive construction, the prefix equalities, the closedness equation, and the QB-8 split.

11.1 The cycle word

For a starting depth c and a period length p, define

$$\text{cycleWord}(c, 0) = [],$$

$$\text{cycleWord}(c, p + 1) = \text{cycleWord}(c, p) + + [v_2(5\,\text{Orbit}_7(c + p) + 1)]\,.$$

The appended entry is the exact step weight at depth $c + p$.

11.2 Prefix orbit equality

Lemma 11.1. *For every c, p,*

$$\text{wordOrbit}(\text{cycleWord}(c, p), \text{Orbit}_7(c)) = \text{Orbit}_7(c + p).$$

Proof. Proceed by induction on p. For $p = 0$, the word is empty and both sides equal $\text{Orbit}_7(c)$. For $p + 1$, write

$$w = \text{cycleWord}(c, p), \qquad t = v_2(5\,\text{Orbit}_7(c + p) + 1)\,.$$

By the induction hypothesis,

$$\text{wordOrbit}(w, \text{Orbit}_7(c)) = \text{Orbit}_7(c + p).$$

Appending t gives

$$\mathrm{wordOrbit}(w{+}{+}[t], \mathrm{Orbit}_7(c)) = \frac{5\,\mathrm{Orbit}_7(c+p)+1}{2^t} = \mathrm{Orbit}_7(c+p+1),$$

by the definition of the accelerated step. □

The Lean statement `cycleWord_prefix_orbit_eq` records this equality.

11.3 Exact step weights

Lemma 11.2. *For every $k < p$,*

$$v_2(5\,\mathrm{wordOrbit}(\mathrm{cycleWord}(c,p)\restriction k, \mathrm{Orbit}_7(c)) + 1) = \mathrm{cycleWord}(c,p)_k.$$

Proof. By the prefix equality, the state after k steps is $\mathrm{Orbit}_7(c+k)$. The k-th entry of the cycle word was defined to be the exact valuation at that state. □

This is `cycleWord_step_exact`. The companion statement `cycleWord_take_eq` records the prefix equality for every truncation of the cycle word.

11.4 Closedness

Assume OrbitCycle(7). There are indices $m < n$ with

$$\mathrm{Orbit}_7(m) = \mathrm{Orbit}_7(n).$$

Put $c = m$ and $p = n - m$. The prefix equality gives

$$\mathrm{wordOrbit}(\mathrm{cycleWord}(c,p), \mathrm{Orbit}_7(c)) = \mathrm{Orbit}_7(n) = \mathrm{Orbit}_7(c),$$

so the cycle word is closed.

11.5 The cycle equation

Let $w = \mathrm{cycleWord}(c,p)$, let $m_0 = \mathrm{Orbit}_7(c)$, and let $T = W(w)$. The word-orbit identity gives

$$2^T\, m_0 = 5^p\, m_0 + A(w).$$

Closedness supplies the left-hand side through the exact step division; the identity itself is the compiled statement `cycleWord_cycle_equation`.

From the equation,

$$2^T = 5^p + \frac{A(w)}{m_0} > 5^p,$$

so

$$T > p \log_2 5.$$

This strict inequality is used by the QB-8 split below.

11.6 The rise/C3 filters

Define

$$\mathrm{rise} = \{t \in w : t < 3\}, \qquad \mathrm{c3} = \{t \in w : t \geq 3\}.$$

Every rise entry is 1 or 2, every C3 entry is at least 3, and the two filters partition the word and its total weight:

$$|\mathrm{rise}| + |\mathrm{c3}| = p, \qquad W(\mathrm{rise}) + W(\mathrm{c3}) = T.$$

11.7 Both filters are nonempty

Proposition 11.3. *For a closed cycle word,* rise *and* c3 *are both nonempty.*

Proof. If c3 were empty, then every entry would be at most 2, so $T \le 2p$. But the cycle equation gives $T > p \log_2 5$, and $\log_2 5 > 2$, contradiction.

If rise were empty, then every entry would be at least 3. For every positive state x and every $t \ge 3$,

$$\frac{5x+1}{2^t} \le \frac{5x+1}{8} < x,$$

because $5x + 1 < 8x$ for $x \ge 1$. Hence every step would strictly decrease the state, and the word could not be closed. □

The Lean records are `cycleQb8Input_hrise_pos`, `cycleQb8Input_hc3_pos`, and `cycleQb8Input_P_ge_two`.

11.8 Rise residues modulo 5

A rise step of weight 1 satisfies

$$2y = 5x + 1,$$

so modulo 5,

$$2y \equiv 1 \pmod 5, \qquad y \equiv 3 \pmod 5.$$

A rise step of weight 2 satisfies

$$4y = 5x + 1,$$

so

$$4y \equiv 1 \pmod 5, \qquad y \equiv 4 \pmod 5.$$

These are the only residue constraints used at the block level; the Lean statement is `rise_step_next_mod_five`.

11.9 The QB-8 input record

The Lean structure `CycleQb8Input m S P w rise c3` records the cycle word together with its filter split, the exact step equations, the closedness, and the positivity and residue data. It is produced from OrbitCycle(7) by `orbit_cycle_imp_cycle_qb8_input`. The extraction is compiled.

11.10 Statement shapes

```
theorem cycleWord_prefix_orbit_eq (c p : Nat) :
    wordOrbit (cycleWord c p) (Orbit 7 c) = Orbit 7 (c + p)

theorem cycleWord_step_exact (c p k : Nat) (hk : k < p) :
    twoValuation
      (5 * wordOrbit ((cycleWord c p).take k) (Orbit 7 c) + 1)
      = (cycleWord c p).getI k

theorem cycleWord_cycle_equation (c p : Nat)
    (hclosed : wordOrbit (cycleWord c p) (Orbit 7 c) = Orbit 7 c) :
    2 ^ wordWeight (cycleWord c p) * Orbit 7 c
      = 5 ^ p * Orbit 7 c + wordA (cycleWord c p)
```

11.11 Status

Statement	Status
cycle-word recursion	compiled
prefix orbit equality	compiled
exact step weights	compiled
cycle equation	compiled
QB-8 split and nonemptiness	compiled
rise residue rules	compiled
cycle-word extraction from OrbitCycle(7)	compiled

12 Corrected window bound architecture

This chapter records the architecture of the corrected decisive window bound: the window values, the corrected predicates, the reachability fields, and the projection from the unified core.

12.1 Window values

The two valuation expressions are

$$\mathrm{t2WindowValue}(j, k_0, \delta, s) = v_2\Big(5^{k_0+1}s + \delta 5^j\Big),$$

$$\mathrm{t1WindowValue}(j, k_0, s) = v_2\Big(5^{k_0+1}s + 5^j - 2\Big).$$

The second expression uses $\delta = 1$, the only value allowed for a weight-1 reset step.

12.2 Corrected predicates

For $t = 2$, the predicate `t2WindowBoundCorrected` asserts

$$a = j - k_0 - 1,\ t = 2,\ \delta \in \{1, 3\},$$
$$\mathrm{ResetWindowReachability}(j, k_0, t, \delta, s) \implies \mathrm{t2WindowValue} \le 2j + 10.$$

For $t = 1$, the predicate `t1WindowBoundCorrected` asserts

$$a = j - k_0 - 1,\ t = 1,$$
$$\mathrm{ResetWindowReachability}(j, k_0, t, 1, s) \implies \mathrm{t1WindowValue} \le 2j + 11.$$

The combined statement `decisiveWindowValuationBoundCorrected` is the conjunction over all parameters.

12.3 Why the arithmetic-only bound is false

The arithmetic-only version drops ResetWindowReachability. It is false: the witness

$$j = 36, \quad k_0 = 0, \quad t = 2, \quad \delta = 3, \quad s = 2^{83} - 3 \cdot 5^{35}$$

satisfies

$$5s + 3 \cdot 5^{36} = 5 \cdot 2^{83},$$

so the valuation is 83, while the arithmetic bound claims at most 82. The witness satisfies the size, parity, and coprimality conditions; it fails only the orbit-reachability fields.

12.4 The reachability fields

The corrected predicate ResetWindowReachability(j, k_0, t, δ, s) carries:

- the previous-terminal equation $s5^{k_0} = r + 1$;
- the general-orbit reachability GeneralOrbitFrom7(r);
- the full-orbit reachability FullOrbitFrom7(r_j);
- the exact reset equation ResetHeadEq$(s, j, k_0, t, \delta, r_j)$;
- $s < 5^{j-1-k_0}$, s odd, and $5 \nmid s$.

The same j, k_0, t appear in every field. A witness that only fixes $j - k_0 - 1$ is weaker and is not accepted.

12.5 Projection from the unified core

The corrected bound is not an independent analytic estimate. The Lean repository records the projection as conditional interfaces:

```
decisiveWindowValuationBoundCorrected_of_unified_core :
  unifiedCoreClosed -> decisiveWindowValuationBoundCorrected

failureWindowExistenceOfUnifiedCore :
  unifiedCoreClosed -> failureWindowExistence
```

Both are compiled. Their common input `unifiedCoreClosed` corresponds to the pending Lean statement `unified_core_final_no_hge`.

12.6 Threshold table

Case	Corrected upper bound	Failure lower bound	Gap
$t = 2$, $\delta \in \{1, 3\}$	$2j + 10$	$2j + 11$	1
$t = 1$	$2j + 11$	$2j + 12$	1

The gap is exactly one in both cases. The proof does not require a stronger bound; it requires only that the reachable upper bound and the block-layer lower bound are incompatible.

12.7 The contradiction

For a reachable failure window with $t = 2$,

$$2j + 11 \le v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \le 2j + 10,$$

which is impossible. For $t = 1$,

$$2j + 12 \le v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \le 2j + 11,$$

also impossible. The packaged lemma is `failure_window_contradicts_window_corrected`.

12.8 Statement shapes

```
def t2WindowBoundCorrected (j k0 a t delta s : Nat) : Prop :=
  a = j - k0 - 1 ->
  t = 2 -> delta = 1 \/ delta = 3 ->
  S6Audit.ResetWindowReachability j k0 t delta s ->
  t2WindowValue j k0 delta s <= 2 * j + 10

def t1WindowBoundCorrected (j k0 a t s : Nat) : Prop :=
  a = j - k0 - 1 -> t = 1 ->
  S6Audit.ResetWindowReachability j k0 t 1 s ->
  t1WindowValue j k0 s <= 2 * j + 11
```

12.9 Status

Statement	Status
window value definitions	compiled
corrected predicates	compiled
reachability predicate definition	compiled
projection interfaces	compiled (conditional)
failure-window contradiction	compiled
closure of the unified core	pending

13 Lean build and reproducibility guide

This chapter records the exact verification setup of the Lean repository. The repository is the final authority for the formal chain; this chapter explains how to reproduce its current state.

13.1 Environment

The project is pinned to

```
Lean 4 : v4.33.0-rc2
Mathlib: locked by the lakefile
```

The dependency cache is populated before the first build. After the cache is present, the build does not require network access.

13.2 Build commands

The main verification targets are:

```
lake build CycleBridge
lake build DwdbDiv
```

A clean checkout must pass both commands. The target `UnifiedCoreAudit` currently contains exactly one `sorry`; the target `FinalStatement` is the final assembly target.

13.3 File map

File	Role
`WordWindow.lean`	words, validity, orbit, molecule
`Pmi.lean`	PMI and PMI-B algebra
`FinitePrefix.lean`	explicit 17-state expansion
`UnifiedCoreBridge.lean`	candidate and segment bridges
`UnifiedCoreAudit.lean`	CRT and final pending inequality
`CycleBridge.lean`	corrected window and no-cycle bridge
`DwdbDiv.lean`	final divergence assembly
`FinalStatement.lean`	final theorem target
`AxiomAudit.lean`	axiom audit

13.4 Import order

```
FinitePrefix.lean
  imports S6AuditStage1

UnifiedCoreBridge.lean
  imports FinitePrefix, UnifiedCoreAudit

UnifiedCoreAudit.lean
  imports S6Audit, S6AuditStage1, LteMacro

CycleBridge.lean
  imports BlockAutomaton, FinalStatement, S6AuditStage1,
          PhOne, SurvExAudit, TdOReal

DwdbDiv.lean
  imports CycleBridge, UnifiedCoreAudit

FinalStatement.lean
  imports S6Audit, FBeta
```

13.5 Axiom audit

The file `AxiomAudit.lean` runs `#print axioms` on each listed bridge theorem. The allowed output is a subset of

```
[propext, Classical.choice, Quot.sound]
```

Any custom `axiom` fails the audit.

13.6 The `sorry` check

The current state of `UnifiedCoreAudit.lean` is exactly one `sorry`, at `unified_core_final_no_hge`. The check is a literal search:

```
grep -n "sorry" UnifiedCoreAudit.lean
```

The result must list exactly that statement.

13.7 The `native_decide` check

The main chain must contain no `native_decide`. The finite base is expanded explicitly:

```
fullOrbitIter 0 = 7 /\ fullOrbitIter 1 = 9 /\
... /\ fullOrbitIter 16 = 34177
```

No code-generation trust boundary is introduced at that step.

13.8 CI

The repository CI runs the two main build targets and the axiom audit on a clean checkout. The CI status is recorded in the repository `README`. The paper does not rely on the CI status alone; the same commands can be run locally.

13.9 Reproducibility

A fresh checkout reproduces the current state with:

```
lake build CycleBridge
lake build DwdbDiv
```

followed by the axiom audit. The pinned toolchain and locked Mathlib dependency make the build deterministic up to the pinned environment.

13.10 Troubleshooting

- If the build fails on a changed file, the status tables in this manual and in the main paper are stale and must not be trusted.
- If `#print axioms` returns an extra axiom, the theorem must be repaired before it is promoted.
- If a second `sorry` appears in the unified-core audit, the ledger must be updated before any status claim is made.
- If `native_decide` appears in a main-chain file, the finite base must be expanded by rewriting instead.

13.11 Acceptance checklist

1. `lake build CycleBridge` passes.
2. `lake build DwdbDiv` passes.
3. No custom axiom appears in the audited theorems.
4. Exactly one `sorry` remains in `UnifiedCoreAudit.lean`.
5. No `native_decide` appears in the main chain.
6. The final theorem is

```
FinalStatement.five_x_plus_one_diverges_at_7 :
  IsUnboundedOrbit 7
```

13.12 Status

Item	Status
main bridge builds	passing
axiom audit	passing for bridge theorems
finite base without `native_decide`	verified by inspection
unified-core final inequality	pending
final theorem assembly	pending

14 Block-head candidate exclusion and the remaining bridge

This chapter consolidates the passage from the block-head candidate to the final valuation inequality. It collects the exact identities that connect the segment exclusions, the CRT candidate, and the terminal residue.

14.1 The block-head candidate

The no-$H_{\geq}$ premises provide a block-head state of the form

$$r = \frac{A_j + 5^j q}{2^{W_j}}.$$

The unified core parameterises its predecessor by

$$x = 2^{e-1} g + \delta 5^{j-1}, \qquad g = \mathrm{Orbit}_7(j-1),$$

with $e \geq 2$ and $\delta \in \{1, 3\}$.

14.2 The q0 interval

The block-head numerator satisfies the exact identity

$$A_j + 5^j q = 2^L \left(B + \delta 5^j\right), \qquad 0 < B < 5^j, \quad A_j < 5^j.$$

Since $A_j < 5^j$, the quotient structure forces

$$q = m + \delta\, 2^L, \qquad m < 2^L.$$

This is `reset_q0_form`. The interval condition

$$2^{W_p} \leq q < 2^{W_j}$$

is equivalent to the block-head bounds

$$5^j \leq 2^t r_j, \qquad r_j < 5^j,$$

by `q0_interval_iff_rj_bound`.

14.3 The block-head identity

Under the reset equation, the same numerator has the block-head form

$$A_j + 5^j q = 2^{W_p} \left(5^{k_0+1} s - 4 + \delta 5^j\right).$$

This is `block_head_identity_of_reset`. It is the exact algebra that connects the block ledger to the reset equation.

14.4 Candidate emptiness

The segment-length exclusions show that no candidate can satisfy the full constraints:

1. $d = 1$ is excluded by the size bound $g < 5^{j-1}/4$;

2. $d = 2$ is excluded by the two congruences modulo 16;

3. $d = 3$ is excluded by the first-big-step contradiction;

4. $d \geq 4$ is excluded by the finite base and the $e = 3, j = 17$ residues.

Hence the candidate family is empty. The corresponding Lean exclusions are compiled.

14.5 From emptiness to the CRT candidate

The exclusion layer leaves the CRT candidate $r_{j,0}$ and the terminal residue y^*. Let

$$n = s - j, \qquad \Delta = W_s - W_j,$$

and let B be the block molecule of the tail of length L. The tail equation is

$$2^{\Delta}\, r_s = 5^n\, r_{j,0} + B.$$

The terminal odd part is

$$w_{\mathrm{term}} = \frac{3r_s + 1}{2^{L+4}},$$

and the terminal identity is

$$2^{\Delta+L+4}\, w_{\mathrm{term}} = 3 \cdot 5^n\, r_{j,0} + 3B + 2^{\Delta}.$$

14.6 The terminal valuation

The final valuation inequality is

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \leq H_s - 2.$$

It fails exactly when

$$w_{\mathrm{term}} \equiv -5^{-(L+3)} \pmod{2^{H_s-1}}.$$

Substituting this failure congruence into the terminal identity gives the CRT equation

$$3 \cdot 5^n\, r_{j,0} + 3B + 2^{\Delta} = 2^{\Delta+L+4} \left(u + m'\, 2^{H_s-1}\right),$$

where u is the least nonnegative residue of $-5^{-(L+3)}$ modulo 2^{H_s-1} and $m' \geq 0$. The terminal residue

$$y^* = -\left(w_{\mathrm{term}} + 5^{-(L+3)}\right) (3 \cdot 5^s)^{-1} \pmod{2^{H_s-1}}$$

is zero exactly in the failure case, so the inequality is equivalent to $y^* \neq 0$.

14.7 The size conditions

The two size conditions are

$$3B + 2^{\Delta} \le 3 \cdot 5^n - 2 \cdot 4^n,$$

and

$$3 \cdot 5^s + (3 \cdot 5^n - 2 \cdot 4^n) \le 2^{\Delta+L+4}\, u.$$

They imply

$$r_{j,0} \ge \frac{3 \cdot 5^s}{3 \cdot 5^n} = 5^j.$$

This is `rj0_ge_of_size_conditions_no_hge`, compiled.

14.8 The remaining bridge

The lower bound $r_{j,0} \ge 5^j$ must be compared with the interval constraints carried by the no-$H_{\ge}$ premises. The comparison is the remaining Lean statement `unified_core_final_no_hge`. It is the single `sorry` in the unified-core audit; the paper does not claim it is closed.

14.9 Statement inventory

Statement	File	Status
`reset_q0_form`	`UnifiedCoreBridge.lean`	compiled
`q0_interval_iff_rj_bound`	`UnifiedCoreBridge.lean`	compiled
`block_head_identity_of_reset`	`UnifiedCoreBridge.lean`	compiled
segment exclusions	`UnifiedCoreBridge.lean`	compiled
`rj0_spec_eq`	`UnifiedCoreAudit.lean`	compiled
`terminal_bound_iff_yStar_ne_zero`	`UnifiedCoreAudit.lean`	compiled
`rj0_ge_of_size_conditions_no_hge`	`UnifiedCoreAudit.lean`	compiled
`unified_core_final_no_hge`	`UnifiedCoreAudit.lean`	pending

14.10 Status

Layer	Status
candidate parameterisation	math proven, Lean compiled
segment exclusions	math proven, Lean compiled
CRT candidate and terminal residue	math proven, Lean compiled
size-condition lower bound	math proven, Lean compiled
final valuation inequality	math proven, Lean pending

15 Finite-prefix verification details

This chapter describes how the finite-prefix base is verified in Lean. The emphasis is on the structure of the verification, not on a transcription of the file.

15.1 The exact expansion

The finite base is the conjunction

$$\mathrm{Orbit}_7(0) = 7,\ \mathrm{Orbit}_7(1) = 9,\ \ldots,\ \mathrm{Orbit}_7(16) = 34177.$$

In Lean this is the statement `fullOrbitIter_prefix_expand`. The proof proceeds by unfolding the orbit definition and rewriting each step with the exact division

$$5\,\mathrm{Orbit}_7(n) + 1 = 2^{t_n}\,\mathrm{Orbit}_7(n+1).$$

15.2 Per-step table

n	$\mathrm{Orbit}_7(n)$	$5\mathrm{Orbit}_7(n)+1$	t_n	$\mathrm{Orbit}_7(n+1)$
0	7	36	2	9
1	9	46	1	23
2	23	116	2	29
3	29	146	1	73
4	73	366	1	183
5	183	916	2	229
6	229	1146	1	573
7	573	2866	1	1433
8	1433	7166	1	3583
9	3583	17916	2	4479
10	4479	22396	2	5599
11	5599	27996	2	6999
12	6999	34996	2	8749
13	8749	43746	1	21873
14	21873	109366	1	54683
15	54683	273416	3	34177

Each row is an exact identity of the form

$$5x + 1 = 2^t y.$$

The verification expands the left-hand side, evaluates the power of two, and checks the quotient; no numerical search is involved.

15.3 Prefix weights

The exact prefix weights are

$$0, 2, 3, 5, 6, 7, 9, 10, 11, 12, 14, 16, 18, 20, 21, 22, 25.$$

They are the partial sums of the exact step weights

$$2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1, 1, 3.$$

The Lean statement `fullOrbit_prefix_step_weights` records the weight word.

15.4 First big step

Lemma 15.1. *For every $n < 15$, $t_n \le 2$, and $t_{15} = 3$.*

Proof. The table gives $t_n \le 2$ for $n < 15$ and $t_{15} = 3$ directly. □

The Lean statement `fullOrbit_first_t_ge3_ is_exactly_3` packages both claims:

```
theorem fullOrbit_first_t_ge3_is_exactly_3 :
    (forall n : Nat, n < 15 ->
      twoValuation (5 * fullOrbitIter n + 1) <= 2) /\
    twoValuation (5 * fullOrbitIter 15 + 1) = 3
```

15.5 Uniqueness

The uniqueness statement is

```
theorem first_big_step_unique (n : Nat)
    (hsmall : forall m : Nat, m < n ->
      twoValuation (5 * fullOrbitIter m + 1) <= 2)
    (hbig : 3 <= twoValuation (5 * fullOrbitIter n + 1)) :
    n = 15 /\ twoValuation (5 * fullOrbitIter n + 1) = 3
```

The proof is a case split on n: for $n < 15$ the hypothesis `hbig` contradicts the table, for $n = 15$ the table gives the weight, and for $n > 15$ the depth-15 step would already be first.

15.6 Why explicit rewriting

The repository records the finite base with explicit rewriting rather than `native_decide`. The purpose is to keep the verification inside the kernel's reduction and rewrite machinery, so no code-generation step is trusted. The paper's audit searches the main-chain files for `native_decide`; the current count is zero.

15.7 Use in the exclusions

The finite base is used in exactly two ways:

- the $d = 3$ family needs a first big step of weight 6, which is excluded because the true first big step has weight 3;
- the $d \geq 4$ branches need a first big step of weight at least 5, also excluded by the same fact.

No later orbit value is used.

15.8 Status

Statement	Status
`fullOrbitIter_prefix_expand`	compiled
`fullOrbit_prefix_step_weights`	compiled
`fullOrbit_first_t_ge3_ is_exactly_3`	compiled
`first_big_step_unique`	compiled
no `native_decide` in the finite base	verified by inspection

16 The divergence bridge in full

This chapter gives the complete divergence bridge as one continuous derivation: from a positive cycle to a contradiction, and then from the absence of cycles to unboundedness.

16.1 Assumption

Assume

$$\mathrm{OrbitCycle}(7).$$

The goal of the bridge is a contradiction. Once the contradiction is established, the general fact

$$\neg\,\mathrm{OrbitCycle}(7) \implies \mathrm{IsUnboundedOrbit}(7)$$

finishes the proof.

16.2 Extraction of the cycle word

There are indices $m < n$ with

$$\mathrm{Orbit}_7(m) = \mathrm{Orbit}_7(n).$$

Put $c = m$, $p = n - m$, and

$$w = \mathrm{cycleWord}(c, p).$$

The cycle word has length p and is valid from $m_0 = \mathrm{Orbit}_7(c)$, and every entry is the exact step weight at the corresponding depth. The extraction is compiled in `orbit_cycle_imp_cycle_qb8_input`.

16.3 The closedness equation

Closedness gives

$$\mathrm{wordOrbit}(w, m_0) = m_0.$$

The word-orbit identity gives the cycle equation

$$2^T m_0 = 5^p m_0 + A(w), \qquad T = W(w).$$

In particular $T > p \log_2 5$.

16.4 The QB-8 split

Filter the word into rise entries and C3 entries:

$$\mathrm{rise} = \{t \in w : t < 3\}, \qquad \mathrm{c3} = \{t \in w : t \geq 3\}.$$

Both filters are nonempty:

- if c3 were empty, then $T \leq 2p$, contradicting $T > p \log_2 5 > 2p$;
- if rise were empty, then every step would satisfy $(5x + 1)/2^t < x$, so the states could not close.

The split is compiled in `CycleQb8Input`.

16.5 The reset block

Choose a rise start. The compiled extraction produces parameters j, k_0, t, δ, s and states r, r_j such that

- $t \in \{1, 2\}$, $\delta = 1$ for $t = 1$, $\delta \in \{1, 3\}$ for $t = 2$;
- $s5^{k_0} = r + 1$;
- GeneralOrbitFrom7(r) and FullOrbitFrom7(r_j);
- ResetHeadEq$(s, j, k_0, t, \delta, r_j)$;
- $s < 5^{j-1-k_0}$, s odd, $5 \nmid s$.

This is the corrected reachability predicate ResetWindowReachability(j, k_0, t, δ, s).

16.6 The block tail and the unified core

The block head r_j is followed by a tail of length L ending at r_s. With $n = s - j$ and $\Delta = W_s - W_j$,

$$2^{\Delta}\, r_s = 5^n\, r_{j,0} + B.$$

The unified core applies to this tail. When its final inequality is closed, the projection interface

`decisiveWindowValuationBoundCorrected_of_unified_core`

produces the corrected window bound. That interface is compiled but consumes `unifiedCoreClosed`, whose Lean representative `unified_core_final_no_hge` is pending.

16.7 The valuation bounds

For $t = 2$, the block-layer balance gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \geq 2j + 11,$$

and the corrected window bound gives

$$v_2\Big(5^{k_0+1}s + \delta 5^j\Big) \leq 2j + 10.$$

For $t = 1$,

$$v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \geq 2j + 12,$$

and

$$v_2\Big(5^{k_0+1}s + 5^j - 2\Big) \leq 2j + 11.$$

Both cases are contradictions.

16.8 The failure window

The construction assembles

$$\mathrm{FailureWindow}(m_0, S, p, w, \mathrm{rise}, \mathrm{c3}, j, k_0, t, \delta, s),$$

whose fields are:

- `hj_lt`: $j < p$;
- `ht`, `hdelta`: the reset weight and parameter;
- `hreset`: the exact reset equation;
- `hreach`: the corrected reachability predicate;
- `hs_odd`, `hs_not_five`, `hk`, `hs_lt`: parity, coprimality, and size;
- `hfail_t1`, `hfail_t2`: the lower bounds.

The existence statement `failureWindowExistence` is pending.

16.9 The contradiction

Given a failure window and the corrected bound, the packaged lemma `failure_window_contradicts_window_corrected` derives `False`. This is compiled.

16.10 No cycle implies unbounded

The general lemma

```
unbounded_of_no_cycle (x : Nat) (h : not OrbitCycle x) :
  IsUnboundedOrbit x
```

is compiled. Its proof uses the pigeonhole principle on the finite set of bounded values.

16.11 The full chain

```
OrbitCycle 7
  -> cycleWord extraction (compiled)
  -> QB-8 split (compiled)
  -> reset block and reachability (compiled)
  -> unified core (unified_core_final_no_hge pending)
  -> corrected window bound (compiled conditional)
  -> lower bound from rise_block_balance (compiled)
  -> FailureWindow existence (pending)
  -> failure_window_contradicts_window_corrected (compiled)
  -> not OrbitCycle 7
  -> IsUnboundedOrbit 7 (assembly target)
```

16.12 Status

Step	Status
cycle-word extraction	compiled
QB-8 split	compiled
reset block and reachability	compiled
block-layer lower bounds	compiled
corrected window projection	compiled (conditional)
failure-window existence	pending
failure-window contradiction	compiled
no-cycle-to-unbounded lemma	compiled
final theorem assembly	pending

17 Rise blocks, C3 blocks, and the block-layer balance

This chapter records the algebra of rise steps, C3 steps, and the block-layer balance that produces the valuation lower bounds.

17.1 Rise steps

A rise step has weight $t = 1$ or $t = 2$.

For $t = 1$,

$$2y = 5x + 1,$$

so modulo 5,

$$2y \equiv 1 \pmod 5, \qquad y \equiv 3 \pmod 5.$$

For $t = 2$,

$$4y = 5x + 1,$$

so

$$4y \equiv 1 \pmod 5, \qquad y \equiv 4 \pmod 5.$$

These are the two residue rules recorded by `rise_step_next_mod_five`.

17.2 C3 steps

A C3 step has weight $t \geq 3$. For every positive state x,

$$\frac{5x+1}{2^t} \leq \frac{5x+1}{8} < x,$$

because $5x + 1 < 8x$ for $x \geq 1$. Hence every C3 step strictly decreases the state. This is the elementary fact `c3Step_lt_of_pos`.

17.3 Small block molecules

For rise blocks, the molecule formulas are:

block word	molecule B	total weight U
$[1]$	1	1
$[2]$	1	2
$[1,1]$	7	2
$[1,2]$	7	3
$[2,1]$	9	3
$[2,2]$	9	4

For example,

$$A([1,2]) = 5 + 2 = 7, \qquad A([2,1]) = 5 + 4 = 9.$$

17.4 A C3 block

The word $[3,1]$ has molecule

$$A([3,1]) = 5 + 8 = 13, \qquad U = 4.$$

Its margins were analysed in the PMI-B chapter: the proper prefix of length 1 is bad.

17.5 Block equation

For any block with weights $u_1, \ldots, u_L$ and total weight U,

$$2^U r_L = 5^L r_0 + B.$$

This is the exact word-orbit identity restricted to the block.

17.6 The reset block

The reset block starts at the candidate predecessor x and maps it to the block head r_j with weight $t \in \{1, 2\}$:

$$2^t\, r_j = 5x + 1.$$

With

$$x = 5^{k_0} s + \delta 5^{j-1} - 1,$$

this becomes

$$t = 2: \quad 4(r_j + 1) = 5^{k_0+1} s + \delta 5^j,$$
$$t = 1: \quad 2(r_j + 1) = 5^{k_0+1} s + 5^j - 2.$$

These are the reset equations used by the corrected window bound.

17.7 The block-layer balance

The compiled lemma `rise_block_balance` derives the valuation lower bounds from the block structure:

$$t = 2: \quad v_2\Big(5^{k_0+1} s + \delta 5^j\Big) \geq 2j + 11,$$
$$t = 1: \quad v_2\Big(5^{k_0+1} s + 5^j - 2\Big) \geq 2j + 12.$$

The lower bounds come from counting the rise and C3 steps in the block and from the exact divisibility of the reset equation. The paper treats the lemma as compiled input; it does not re-derive it in the main text.

17.8 Why the lower bounds are sharp

The arithmetic counterexample

$$(j, k_0, t, \delta, s) = (36, 0, 2, 3, 2^{83} - 3 \cdot 5^{35})$$

attains valuation $83 = 2 \cdot 36 + 11$, exactly the lower bound of the $t = 2$ balance. The corrected window bound would require at most 82. This is the sharpness point: the block-layer lower bound and the reachable upper bound are separated by exactly one.

17.9 Status

Statement	Status
rise residue rules	compiled
C3 decrease fact	compiled
block molecule formulas	math proven
reset equations	compiled
`rise_block_balance`	compiled
corrected window bound	compiled (conditional)

18 Glossary, notation, and reading guide

This chapter collects the notation and terminology used in the paper and in this manual, and gives reading routes for different audiences.

18.1 Mathematical symbols

Symbol	Meaning
$T(x)$	accelerated step $(5x+1)/2^{v_2(5x+1)}$
$\mathrm{Orbit}_x(n)$	n-th accelerated orbit value of x
t_j	exact step weight at depth j
W_j	accumulated weight before step j
w	a word of step weights
$A(w)$	word molecule of w
m_j	margin $j\log_2 5 - W_j$
α	$\log_2 5$
C_1, C_2, C_3	counts of weights 1, 2, at least 3
g	orbit state $\mathrm{Orbit}_7(j-1)$
x	candidate predecessor
r_j	block head
r_s	terminal state
B	block molecule
u_i	segment step weights
W_i	segment prefix weights
H_s	unified-core threshold parameter

18.2 Reset-window symbols

Symbol	Meaning
j	reset step depth
k_0	exponent in $s5^{k_0} = r+1$
t	reset step weight, $t \in \{1,2\}$
δ	reset parameter: 1 for $t=1$, $\{1,3\}$ for $t=2$
s	odd terminal part with $5 \nmid s$
r	previous terminal state
n	tail length $s-j$
Δ	weight difference $W_s - W_j$

18.3 Status terms

- **Math: proven**: the mathematical argument is written out.
- **Lean: compiled**: the Lean theorem compiles without `sorry`.
- **Lean: pending**: the Lean theorem is not yet closed.
- **compiled (conditional)**: the theorem compiles but consumes an open input.

18.4 Reading routes

For a mathematician. Read the main paper in order: definitions, framework, unified core, bridge, Lean. Use the supplementary chapters for the full derivations: the block calculus chapter, the unified-core chapter, the CRT chapter, and the divergence-bridge chapter.

For a Lean reader. Begin with the Lean theorem index and the reproducibility guide, then read the statement shapes in the status matrix. Follow the import order: `WordWindow`, `Pmi`, `FinitePrefix`, `UnifiedCoreBridge`, `UnifiedCoreAudit`, `CycleBridge`, `DwdbDiv`.

For a referee. Check the pending ledger first. Two statements are pending: `unified_core_final_no_hge` and `five_x_plus_one_diverges_at_7`. Then audit the dependency graph to confirm that every compiled claim has a build.

18.5 Cross-reference map

Main-paper section	Supplementary chapters
introduction and main theorem	glossary, status matrix
string-flow framework	vocabulary, PMI-B, block calculus
unified core	unified core, CRT, candidate exclusion
divergence bridge	cycle word algebra, window architecture, divergence bridge in full
Lean verification	Lean theorem index, reproducibility guide
appendix	all chapters

18.6 How to use the manual

1. Start with the glossary to fix notation.
2. Follow the reading route for your audience.
3. Use the status matrix for any Lean claim.
4. Use the dependency graph to find what a statement needs.
5. Never cite a pending statement as closed.

19 Proof architecture and dependency diagrams

This chapter records the architecture of the proof as diagrams. The diagrams are schematic: they show dependencies and statuses, not proof terms.

19.1 The layer diagram

```
Layer 1: string-flow ledger
  word, validity, weight, molecule, PMI

Layer 2: finite base
  explicit 17 states, first big step

Layer 3: unified core
  candidate parameterisation
  segment exclusions d=1, d=2, d=3, d>=4
  CRT candidate and terminal residue
  final valuation inequality (pending in Lean)

Layer 4: cycle bridge
  cycle word extraction
  QB-8 split
  corrected window bound
  failure window contradiction
```

```
Layer 5: final assembly
  no cycle -> unbounded
  IsUnboundedOrbit 7 (pending)
```

19.2 File import graph

```
FinitePrefix.lean
  imports S6AuditStage1

UnifiedCoreBridge.lean
  imports FinitePrefix, UnifiedCoreAudit

UnifiedCoreAudit.lean
  imports S6Audit, S6AuditStage1, LteMacro

CycleBridge.lean
  imports BlockAutomaton, FinalStatement, S6AuditStage1,
          PhOne, SurvExAudit, Td0Real

DwdbDiv.lean
  imports CycleBridge, UnifiedCoreAudit

FinalStatement.lean
  imports S6Audit, FBeta
```

19.3 Conditional interface graph

```
unifiedCoreClosed (open)
  -> decisiveWindowValuationBoundCorrected_of_unified_core
       (compiled conditional)
  -> failureWindowExistenceOfUnifiedCore
       (compiled conditional)
  -> failureWindowExistence (open)
  -> windowBoundToNoCycle_of_failureWindowExistence
       (compiled conditional)
  -> dwdbDivFinalAssembly (compiled conditional)
  -> five_x_plus_one_diverges_at_7 (pending)
```

19.4 Topological theorem order

1. word-orbit identity;
2. PMI and PMI-B;
3. finite prefix and first-big-step facts;
4. reset-head predecessor and candidate parameterisation;
5. segment equations and exclusions;

6. CRT candidate, terminal residue, size lower bound;
7. final valuation inequality (pending);
8. cycle-word extraction and QB-8 split;
9. reset block and reachability;
10. corrected window bound (conditional);
11. failure-window contradiction;
12. no-cycle-to-unbounded lemma;
13. final theorem (pending).

19.5 Status flow

```
compiled facts
  -> compiled facts
  -> unified core (one pending Lean statement)
  -> conditional bridge interfaces
  -> final assembly (pending)
```

19.6 What the diagrams do not show

The diagrams show the dependency structure, not the proof size or the time taken to check a statement. A node marked compiled may depend on hundreds of Mathlib lemmas; the diagrams record only the project-level dependencies that the manual tracks.

19.7 Status

Node	Status
layer 1–3 mathematics	proven
layer 3 Lean final inequality	pending
layer 4 cycle bridge	compiled (conditional)
layer 5 final assembly	pending

20 Why the proof is exact

This chapter explains why the proof uses exact identities rather than estimates, probability, or transcendence statements.

20.1 The exact ledger

The core identity is

$$2^{W(w)}\,\mathrm{wordOrbit}(w, x_0) = 5^{\mathrm{len}(w)}\,x_0 + A(w).$$

Every later equation in the proof is a projection of this identity: block equations, segment equations, reset equations, and the CRT equation. There is no place where a size estimate replaces an equality.

20.2 Where loose estimates would fail

An average-growth estimate would conclude that the orbit should increase on average, which is compatible with both unboundedness and with a long excursion followed by a cycle. The proof needs the opposite: a contradiction that is sensitive to a single unit of 2-adic valuation. The decisive-window thresholds are separated by exactly one:

$$2j + 10 < 2j + 11 < 2j + 12.$$

An estimate that is off by one cannot distinguish the reachable upper bound from the block-layer lower bound.

20.3 The counterexample as a failure mode

The arithmetic witness

$$s = 2^{83} - 3 \cdot 5^{35}$$

satisfies every size, parity, and coprimality condition used by the old bound, but its valuation is 83, one more than 82. The witness shows exactly what an arithmetic-only argument can miss: the reachability fields. No estimate can repair this, because the failure is an exact valuation mismatch.

20.4 What probability cannot provide

A probabilistic model can predict the distribution of step weights, but the proof requires a statement about the specific orbit of 7. Equidistribution would not rule out a cycle with an exceptional valuation profile; the proof must rule out every cycle by exact arithmetic.

20.5 What transcendence cannot provide

The margin $\log_2 5$ is irrational, but the proof never needs irrationality or transcendence. The PMI identity is cleared of denominators:

$$\sum_{j=0}^{L-1} 2^{W_j}\, 5^{L-j} = 5\, A(w).$$

All inequalities are between integers or between rationals with explicit denominators. Transcendence estimates would be both too weak and unnecessary.

20.6 The role of the finite base

The only finite data are the first 17 states of the orbit of 7. The finite base is used to identify the first big step; every later claim is exact arithmetic. The base is expanded explicitly in Lean, so no search or code generation is trusted.

20.7 Exactness checklist

- every block equation is the word-orbit identity;
- every segment exclusion uses exact equations and exact congruences;
- the CRT candidate is the least solution of an exact linear equation;

- the size conditions are exact inequalities;
- the window thresholds are exact and never weakened;
- the pending Lean statement is the only unverified node.

20.8 Status

Item	Status
exact ledger identities	compiled
finite base	compiled
segment and modular exclusions	compiled
CRT layer	compiled except the final assembly
decisive-window thresholds	unchanged

21 Full derivation of the $d = 2$ exclusion

This chapter gives the complete derivation of the $d = 2$ exclusion, including the size analysis, the modular arithmetic, and the final contradiction.

21.1 The segment equation

For $d = 2$, the word molecule is

$$A(w) = 5 + 2^{u_1}, \qquad W = u_1 + u_2,$$

and the segment equation is

$$\left(2^{u_1+e} - 25\right) g + 2^{u_1+1} \delta 5^{j-1} = 5 + 2^{u_1}.$$

21.2 The size restriction

For a rise segment, $u_1 \in \{1, 2\}$, so the right-hand side is at most

$$5 + 2^2 = 9.$$

The left-hand side contains the positive term

$$2^{u_1+1} \delta 5^{j-1} \geq 2 \cdot 5^{j-1},$$

so the coefficient $2^{u_1+e} - 25$ must be negative and small enough to cancel it. In particular

$$2^{u_1+e} < 25.$$

With $e \geq 2$ and $u_1 \in \{1, 2\}$, the only possibilities are

$$(u_1, e) \in \{(1, 2), (1, 3), (2, 2)\}.$$

21.3 Case $u_1 = 1$, $e = 2$

The equation becomes

$$17g = 4\delta 5^{j-1} - 7.$$

The size bound is

$$g < \frac{5^{j-1}}{2}.$$

For $\delta = 3$,

$$g = \frac{12 \cdot 5^{j-1} - 7}{17}.$$

This exceeds $5^{j-1}/2$ because

$$24 \cdot 5^{j-1} - 14 > 17 \cdot 5^{j-1}$$

is equivalent to $7 \cdot 5^{j-1} > 14$, which holds for $j \geq 3$. Hence only $\delta = 1$ survives:

$$17g = 4 \cdot 5^{j-1} - 7.$$

21.4 Case $u_1 = 1$, $e = 3$

The equation becomes

$$9g = 8\delta 5^{j-1} - 7.$$

The size bound is the stronger

$$g < \frac{5^{j-1}}{4}.$$

Even for $\delta = 1$,

$$g = \frac{8 \cdot 5^{j-1} - 7}{9} > \frac{5^{j-1}}{4},$$

because $32 \cdot 5^{j-1} - 28 > 9 \cdot 5^{j-1}$. This case is excluded.

21.5 Case $u_1 = 2$, $e = 2$

The equation becomes

$$9g = 8\delta 5^{j-1} - 9.$$

The size bound is

$$g < \frac{5^{j-1}}{2}.$$

Even for $\delta = 1$,

$$g = \frac{8 \cdot 5^{j-1} - 9}{9} > \frac{5^{j-1}}{2},$$

because $16 \cdot 5^{j-1} - 18 > 9 \cdot 5^{j-1}$. This case is excluded.

21.6 The survivor

The only surviving branch is

$$\delta = 1, \qquad e = 2, \qquad u_1 = 1,$$

with equation

$$17g = 4 \cdot 5^{j-1} - 7.$$

21.7 Congruence modulo 17

Modulo 17,

$$4 \cdot 5^{j-1} \equiv 7.$$

Since $4^{-1} \equiv 13 \pmod{17}$,

$$5^{j-1} \equiv 7 \cdot 13 = 91 \equiv 6 \pmod{17}.$$

The powers of 5 modulo 17 have order 16, and

$$5^3 = 125 \equiv 6 \pmod{17},$$

so

$$j - 1 \equiv 3 \pmod{16}.$$

21.8 The candidate congruence

The candidate is

$$x = 2g + 5^{j-1}.$$

Eliminating g from $17g = 4 \cdot 5^{j-1} - 7$ gives

$$17x = 25 \cdot 5^{j-1} - 14.$$

The orbit-level candidate satisfies

$$x \equiv 743 \pmod{1280}.$$

Multiplying by 17,

$$17 \cdot 743 = 12631 \equiv 1111 \pmod{1280},$$

so

$$25 \cdot 5^{j-1} - 14 \equiv 1111 \pmod{1280},$$

hence

$$25 \cdot 5^{j-1} \equiv 1125 \pmod{1280}.$$

Dividing by 5 and then by 5 modulo 256,

$$5^{j-1} \equiv 45 \pmod{256}.$$

21.9 Congruence modulo 256

Since

$$5^7 = 78125 \equiv 45 \pmod{256}$$

and the order of 5 modulo 256 is 64,

$$j - 1 \equiv 7 \pmod{64}.$$

21.10 The contradiction

The congruence $j - 1 \equiv 3 \pmod{16}$ and the congruence $j - 1 \equiv 7 \pmod{64}$ contradict each other modulo 16:

$$3 \not\equiv 7 \pmod{16}.$$

Therefore $d = 2$ is impossible.

21.11 Lean statement

The exclusion is

```
theorem d2_exclusion
    (j e g delta u1 : Nat)
    (hj : 3 <= j)
    (hg : g < 5 ^ (j - 1) / 2 ^ (e - 1))
    (hdelta : delta = 1 or delta = 3)
    (he : 2 <= e)
    (hu1 : u1 = 1 or u1 = 2)
    (hseg : 2 ^ (u1 + e) * g + 2 ^ (u1 + 1) * delta * 5 ^ (j - 1)
      = 5 + 2 ^ u1 + 25 * g) :
    False
```

21.12 Status

Item	Status
size analysis	math proven
modular contradiction	math proven
`d2_exclusion`	Lean compiled
`d2_exclusion_of_orbit`	Lean compiled

22 Full derivation of the $d = 3$ exclusion

This chapter gives the complete derivation of the $d = 3$ exclusion: the surviving family, its state formula, its residue class, and the finite-base contradiction.

22.1 The segment equation

For $d = 3$, the word molecule is

$$A(w) = 25 + 5 \cdot 2^{u_1} + 2^{u_1+u_2}, \qquad W = u_1 + u_2 + u_3.$$

The segment equation is

$$2^{W+e-1}g + 2^W \delta 5^{j-1} = 5^3 g + 25 + 5 \cdot 2^{u_1} + 2^{u_1+u_2}.$$

22.2 The surviving family

The branch analysis reduces the parameters to

$$t_j = 2, \qquad \delta = 1, \qquad e = 2, \qquad u_1 = u_2 = 1, \qquad a = 0,$$

which forces

$$u_3 = 1.$$

Then

$$A(w) = 25 + 10 + 4 = 39, \qquad W = 3.$$

The segment equation becomes

$$2^{3+1}g + 2^3 \cdot 5^{j-1} = 5^3 g + 39,$$

i.e.

$$16g + 8 \cdot 5^{j-1} = 125g + 39.$$

Rearranging,

$$109g = 8 \cdot 5^{j-1} - 39,$$

so

$$g = \frac{8 \cdot 5^{j-1} - 39}{109}.$$

22.3 The integrality condition

The state g is an integer, so

$$8 \cdot 5^{j-1} \equiv 39 \pmod{109}.$$

The inverse of 8 modulo 109 is 41, because

$$8 \cdot 41 = 328 = 3 \cdot 109 + 1.$$

Therefore

$$5^{j-1} \equiv 39 \cdot 41 = 1599 \equiv 73 \pmod{109},$$

since $1599 - 14 \cdot 109 = 73$.

22.4 The residue class

The full family calculation combines this congruence with the remaining branch conditions and produces

$$j - 1 \equiv 923 \pmod{1728}.$$

The modulus 1728 is specific to the $d = 3$ family calculation; the paper records the final class and the consistency checks below.

22.5 Consistency checks

Repeated squaring modulo 109 gives:

k	$5^k \bmod 109$	k	$5^k \bmod 109$	k	$5^k \bmod 109$
1	5	16	89	256	26
2	25	32	73	512	22
4	80	64	97	923	73
8	78	128	35		

The entry $5^{923} \equiv 73$ follows from

$$923 = 512 + 256 + 128 + 16 + 8 + 2 + 1$$

and the corresponding product of residues. Finally,

$$8 \cdot 73 = 584 \equiv 39 \pmod{109},$$

so the representative 923 is consistent with the integrality of g.

22.6 The first big step

For the surviving family, the segment weights are

$$u_1 = u_2 = u_3 = 1,$$

and the first step of weight at least 3 occurs at depth $j+4$ with weight 6. The finite base says that the first big step of the orbit of 7 occurs at depth 15 and has weight exactly 3. Hence the family contradicts the finite base.

22.7 Lean statement

The exclusion is compiled as `d3_family_big_weight_excluded`. It uses the finite-prefix facts `fullOrbit_first_t_ge3_ is_exactly_3` and `first_big_step_unique`.

22.8 Status

Item	Status
family reduction	math proven
state formula and integrality	math proven
residue class and consistency	math proven
finite-base contradiction	math proven
`d3_family_big_weight_excluded`	Lean compiled

23 Full derivation of the $d \geq 4$ exclusions

This chapter gives the complete derivation of the $d \geq 4$ exclusions. The proof splits according to the location and weight of the first big step.

23.1 The three branch types

For $d \geq 4$, the first big step of the candidate block is one of three types:

1. the incoming step $g_{\text{prev}} \to g$, which happens when $e \geq 3$;
2. the step $y \to x$, which happens when $e = 2$ and $a \geq 1$ and has weight $1 + 4a \geq 5$;
3. the step $z \to w$, which happens when $e = 2$ and $a = 0$ and has weight 5 or 6.

23.2 The finite base

The finite base says:

- for every $n < 15$, $t_n \leq 2$;
- $t_{15} = 3$;
- the first big step is unique.

These facts are compiled in `fullOrbit_first_t_ge3_ is_exactly_3` and `first_big_step_unique`.

23.3 Branch $e \geq 3$

The incoming step $g_{\text{prev}} \to g$ has weight $e \geq 3$, so it is a big step. The finite base forces

$$e = 3, \qquad j = 17.$$

For this branch the candidate residue is then excluded modulo 640 or 1280. The two compiled statements are `dge4_e3_j17_t1_excluded` and `dge4_e3_j17_t2_delta3_excluded`.

23.4 Branch $e = 2$, $a \geq 1$

The step $y \to x$ has weight

$$1 + 4a \geq 5.$$

This would be the first big step of the orbit prefix. But the finite base says the first big step has weight exactly 3, so this branch is impossible. The compiled statement is `dge4_e2_a_ge1_excluded`.

23.5 Branch $e = 2$, $a = 0$

The step $z \to w$ has weight 5 or 6, again contradicting the finite base. The same uniqueness argument applies: the true first big step occurs at depth 15 with weight 3, so a later candidate big step cannot be first, and an earlier one cannot have weight 5 or 6.

23.6 Why uniqueness is used

The uniqueness statement says that any depth whose earlier weights are all at most 2 and whose own weight is at least 3 must be depth 15 with weight 3. A candidate first big step of weight at least 5 fails this conclusion, regardless of where it is placed.

23.7 Compiled statements

- `dge4_e2_a_ge1_ excluded`;
- `dge4_e3_j17_t1_excluded`;
- `dge4_e3_j17_t2_delta3_excluded`;
- `candidate_first_big_step_weight_ge_five`.

23.8 Status

Item	Status
branch classification	math proven
$e = 2$ branches	math proven, Lean compiled
$e = 3$, $j = 17$ residues	math proven, Lean compiled

24 Finite-prefix proof script outline

This chapter outlines how the finite-prefix base is organised as a proof script in Lean. The outline describes the structure of the proof; the repository file is the authoritative source.

24.1 The goal

The script proves the conjunction

$$\mathrm{Orbit}_7(0) = 7 \wedge \mathrm{Orbit}_7(1) = 9 \wedge \cdots \wedge \mathrm{Orbit}_7(16) = 34177.$$

In Lean this is

```
fullOrbitIter_prefix_expand :
  fullOrbitIter 0 = 7 /\ fullOrbitIter 1 = 9 /\
  ... /\ fullOrbitIter 16 = 34177
```

24.2 The unfolding

The orbit definition unfolds each value:

$$\mathrm{Orbit}_7(n+1) = \frac{5\,\mathrm{Orbit}_7(n) + 1}{2^{t_n}}.$$

The script rewrites each quotient using the exact division

$$5\,\mathrm{Orbit}_7(n) + 1 = 2^{t_n}\,\mathrm{Orbit}_7(n+1).$$

24.3 Exact division table

n	$\mathrm{Orbit}_7(n)$	$5\mathrm{Orbit}_7(n)+1$	t_n	quotient
0	7	36	2	9
1	9	46	1	23
2	23	116	2	29
3	29	146	1	73
4	73	366	1	183
5	183	916	2	229
6	229	1146	1	573
7	573	2866	1	1433
8	1433	7166	1	3583
9	3583	17916	2	4479
10	4479	22396	2	5599
11	5599	27996	2	6999
12	6999	34996	2	8749
13	8749	43746	1	21873
14	21873	109366	1	54683
15	54683	273416	3	34177

24.4 Weight facts

The script derives the weight word

$$2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1, 1, 3$$

and then proves the first-big-step conjunction:

```
theorem fullOrbit_first_t_ge3_is_exactly_3 :
    (forall n : Nat, n < 15 ->
      twoValuation (5 * fullOrbitIter n + 1) <= 2) /\
    twoValuation (5 * fullOrbitIter 15 + 1) = 3
```

24.5 Uniqueness

The uniqueness lemma is proved by case analysis on n:

- for $n < 15$, the small-weight facts contradict the big-weight hypothesis;
- for $n = 15$, the table gives weight 3;
- for $n > 15$, the depth-15 step would already be first.

24.6 No `native_decide`

The finite base deliberately avoids `native_decide`. Every quotient is rewritten from an explicit equality, so the proof stays inside the kernel's reduction and rewriting machinery. The audit searches the main-chain files for `native_decide`; the count is zero.

24.7 Verification checklist

1. `fullOrbitIter_prefix_expand` compiles;
2. `fullOrbit_prefix_step_weights` compiles;
3. `fullOrbit_first_t_ge3_is_exactly_3` compiles;
4. `first_big_step_unique` compiles;
5. no `native_decide` appears in the file.

24.8 Status

Item	Status
explicit expansion	compiled
step weights	compiled
first-big-step facts	compiled
uniqueness	compiled
no `native_decide`	verified by inspection

25 The CRT layer: statement-by-statement commentary

This chapter comments on each statement in the CRT layer of `UnifiedCoreAudit.lean`. The statements are recorded together with their mathematical role and their current status.

25.1 `All36_20PremisesNoHge`

This is the record of the no-$H_{\geq}$ premises. It carries the block-head data

$$j,\ W_p,\ W_j,\ q,\ A_j,\ A_s,\ s,\ W_s,\ r_s,\ L,\ H_s$$

and the step-weight function weight. The record is a definition, not a theorem; it is compiled.

25.2 `blockB_bound_of_no_hge`

This statement derives the block-tail bound

$$3B + 2^{\Delta} \le 3 \cdot 5^{n} - 2 \cdot 4^{n},$$

where $n = s - j$ and $\Delta = W_s - W_j$. It is the first size condition used by the CRT argument. The statement is compiled.

25.3 `rj0_spec_eq`

This statement packages the CRT equation

$$3 \cdot 5^{n}\, r_{j,0} + 3B + 2^{\Delta} = 2^{\Delta+L+4}\left(u + m'\, 2^{H_s-1}\right)$$

and the definition of the candidate $r_{j,0}$ as its least nonnegative solution. The statement is compiled.

25.4 `terminal_bound_iff_yStar_ne_zero`

This statement records the equivalence

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \le H_s - 2 \iff y^{*} \neq 0.$$

The residue y^{*} is defined by

$$y^{*} = -\left(w_{\mathrm{term}} + 5^{-(L+3)}\right)(3 \cdot 5^{s})^{-1} \pmod{2^{H_s-1}}.$$

The statement is compiled.

25.5 `rj0_ge_of_size_conditions_no_hge`

This statement derives

$$r_{j,0} \ge 5^{j}$$

from the two size conditions. The derivation is the inequality chain recorded in the main paper. The statement is compiled.

25.6 `unified_core_final_no_hge`

This is the final valuation inequality

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \le H_s - 2$$

under the no-$H_{\ge}$ premises, the block-head equality, the full-orbit reachability of r, and $2 \le H_s$. The statement is the unique `sorry` in the unified-core audit and is pending.

25.7 How the layer fits

```
premises record
  -> block tail bound (compiled)
  -> CRT equation and candidate (compiled)
  -> terminal residue equivalence (compiled)
  -> size-condition lower bound (compiled)
  -> final valuation inequality (pending)
```

25.8 Statement shapes

```
theorem terminal_bound_iff_yStar_ne_zero
    (L H_s s r_s : Nat) (hH : 2 <= H_s) :
    twoValuation (5 ^ (L + 3) * wTerminal L r_s + 1) <= H_s - 2 <->
      yStar L H_s s r_s != 0

theorem rj0_ge_of_size_conditions_no_hge
    (j Wp Wj q Aj A_s s W_s r_s L H_s : Nat)
    (weight : Nat -> Nat)
    (hPrem : All36_20PremisesNoHge
       j Wp Wj q Aj A_s s W_s r_s L H_s weight)
    (hH : 1 <= H_s)
    (hBase2 : 3 * 5 ^ s
      + (3 * 5 ^ (s - j) - 2 * 4 ^ (s - j))
      <= 2 ^ (W_s - Wj + L + 4) * uResidue L (H_s - 1))
    (hPow3 : 3 * 5 ^ s
      + (3 * 5 ^ (s - j) - 2 * 4 ^ (s - j))
      <= 2 ^ (W_s - Wj + L + H_s + 3)) :
    5 ^ j <= rj0 j Wp Wj q Aj A_s s W_s r_s L H_s weight
```

25.9 Status

Statement	Status
premises record	compiled
block-tail bound	compiled
CRT equation and candidate	compiled
terminal residue equivalence	compiled
size-condition lower bound	compiled
final valuation inequality	pending

26 The corrected reachability predicate

This chapter explains the corrected reachability predicate ResetWindowReachability(j, k_0, t, δ, s) field by field, and why each field is necessary.

26.1 The fields

The predicate ResetWindowReachability(j, k_0, t, δ, s) asserts the existence of states r and r_j such that:

1. $s5^{k_0} = r + 1$;
2. GeneralOrbitFrom7(r);
3. FullOrbitFrom7(r_j);
4. ResetHeadEq$(s, j, k_0, t, \delta, r_j)$;
5. $s < 5^{j-1-k_0}$;

6. s is odd;

7. $5 \nmid s$.

The first four items are reachability data; the last three are size and parity conditions.

26.2 Why the same indices

The predicate fixes the same j, k_0, t in every field. The reset equation, the size bound, and the two reachability predicates all use the same depth and weight. This prevents a witness from combining a reset equation at one depth with a size bound at another.

26.3 A weaker predicate would fail

A predicate that only fixed the difference $j - k_0 - 1$ would allow a reset equation with different absolute indices than the reachability witness. The corrected predicate is written with the absolute indices precisely to block that gap.

26.4 The arithmetic witness

The witness

$$j = 36, \quad k_0 = 0, \quad t = 2, \quad \delta = 3, \quad s = 2^{83} - 3 \cdot 5^{35}$$

satisfies the size and parity conditions:

- $s < 5^{35}$ because $2^{81} < 5^{35}$;
- s is odd;
- $s \equiv 3 \pmod 5$, so $5 \nmid s$.

It also satisfies the valuation identity

$$5s + 3 \cdot 5^{36} = 5 \cdot 2^{83},$$

which is why it breaks the arithmetic-only bound.

26.5 What the witness lacks

The witness supplies no r with

$$s5^0 = r + 1,$$

no general-orbit reachability GeneralOrbitFrom7(r), no full-orbit reset head FullOrbitFrom7(r_j), and no reset equation

$$\text{ResetHeadEq}(s, 36, 0, 2, 3, r_j).$$

It is arithmetic-only, so it is not a counterexample to the corrected bound.

26.6 Statement shape

The repository definition is `S6Audit.ResetWindowReachability`. The paper writes it schematically as an existence predicate over r and r_j with the seven fields listed above; the precise record structure lives in the repository and may use different field names. The mathematical content is the seven fields, not the layout of the record.

26.7 Status

Item	Status
predicate definition	compiled
arithmetic witness	verified exactly
corrected bound using the predicate	compiled (conditional)
failure-window construction	pending

27 Worked examples of the exact ledger

This chapter works through concrete examples of the exact ledger. The examples are not proofs of the theorem; they illustrate the algebra that the theorem uses.

27.1 The orbit word

The exact prefix word of the orbit of 7 is

$$[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1, 1, 3].$$

Its total weight is

$$T = 25,$$

and the final prefix identity is

$$2^{25} \cdot 34177 = 5^{16} \cdot 7 + 78674588089.$$

27.2 Block equations

$$\begin{aligned} [2,1] \text{ from } 7: &\qquad 2^3 \cdot 23 = 184 = 5^2 \cdot 7 + 9, \\ [1,2] \text{ from } 9: &\qquad 2^3 \cdot 29 = 232 = 5^2 \cdot 9 + 7, \\ [1,1,2] \text{ from } 29: &\qquad 2^4 \cdot 229 = 3664 = 5^3 \cdot 29 + 39. \end{aligned}$$

27.3 PMI sum

For $w = [2, 1, 2]$ from 7,

$$\sum_{j=0}^{2} 2^{-m_j} = 1 + \frac{4}{5} + \frac{8}{25} = \frac{53}{25} = \frac{5A(w)}{5^3}.$$

All three proper prefixes have positive margin.

27.4 Why closure is restrictive

Consider the word

$$w = [2, 2, 3].$$

Its molecule is

$$A(w) = 25 + 5 \cdot 4 + 2^{2+2} = 25 + 20 + 16 = 61,$$

and its total weight is 7. If it were closed from a state m, then

$$2^7 \, m = 5^3 \, m + 61,$$

so

$$128m = 125m + 61, \qquad 3m = 61,$$

which has no integer solution. The word satisfies the weight inequality $T > 3\log_2 5$ but still cannot close.

27.5 A longer example

For

$$w = [3, 3, 2, 2],$$

the molecule is

$$A(w) = 125 + 200 + 320 + 256 = 901,$$

and the total weight is 10. Closedness would require

$$2^{10}\, m = 5^4\, m + 901,$$

so

$$1024m = 625m + 901, \qquad 399m = 901,$$

again with no integer solution.

27.6 Insufficient weight

For

$$w = [2, 2, 2, 2, 3],$$

the molecule is

$$A(w) = 625 + 500 + 400 + 320 + 256 = 2101,$$

and the total weight is 11. Since

$$11 < 5\log_2 5 \approx 11.6096,$$

closedness is impossible already by the weight inequality: the left-hand side $2^{11}m$ is smaller than $5^5 m$ for every positive m.

27.7 The lesson

These examples show why the cycle equation is restrictive: the weights must satisfy a strict margin inequality, and the molecule must also divide the weight difference

$$A(w) = \left(2^T - 5^P\right) m.$$

The proof uses this exact equation to extract every constraint on a cycle word.

27.8 Status

Item	Status
orbit prefix identities	compiled
block equations	math proven
PMI sums	compiled
closure examples	exact arithmetic, not proof claims

28 Submission and release checklist

This chapter records the checks that must pass before the manuscript is submitted or the final theorem is promoted.

28.1 Document checks

- `main.tex` compiles with `pdflatex`, `bibtex`, and two further passes.
- `supplement_manual.tex` compiles with three passes.
- Neither log contains overfull boxes, undefined references, or warnings that change the meaning of a statement.
- The main document is between 40 and 70 pages.
- The supplementary manual is between 100 and 150 pages.

28.2 Discipline checks

- The constant 13 and the thresholds +10, +11, +12 are unchanged.
- The phrases "independent analytic gap" and its Chinese equivalent do not appear.
- The phrase "automatic" is not used to assert a proof step.
- The pending statements `unified_core_final_no_hge` and `five_x_plus_one_diverges_at_7` are never marked compiled or closed.
- The notation t_j and W_{j-1} are never interchanged.

28.3 Lean checks

- `lake build CycleBridge` passes.
- `lake build DwdbDiv` passes.
- `#print axioms` on the bridge theorems returns only `propext`, `Classical.choice`, and `Quot.sound`.
- `UnifiedCoreAudit.lean` contains exactly one `sorry`.
- The main chain contains no `native_decide`.

28.4 Repository checks

- The anonymous repository is `a-stringflow/stringflow-proof`.
- The repository README records the AI assistance disclosure.
- The CI status for the main build targets is recorded.
- The ledgers in `docs/` are current.

28.5 Release gate

The final theorem

```
FinalStatement.five_x_plus_one_diverges_at_7 :
  IsUnboundedOrbit 7
```

may be promoted only after:

1. `unified_core_final_no_hge` is closed in Lean;
2. the final theorem is assembled and compiles;
3. every document check above is re-run;
4. every status table is updated.

28.6 Checklist table

Item	Status
main document length	43 pages
supplementary manual length	102 pages
document compile	passing
discipline checks	passing
main bridge builds	passing
axiom audit	passing
unified-core final inequality	pending
final theorem assembly	pending

29 Lean theorem index

This chapter gives a theorem-by-theorem index of the main files.

29.1 FinitePrefix.lean

- `fullOrbitIter_prefix_expand`
- `fullOrbit_prefix_step_weights`
- `fullOrbit_first_t_ge3_is_exactly_3`
- `first_big_step_unique`
- `candidate_first_big_step_weight_ne_three`
- `candidate_first_big_step_weight_ge_five`

29.2 UnifiedCoreBridge.lean

- `candidateX`, `candidateRj`, `orbitState`
- `reset_head_predecessor`
- `candidateX_of_reset_and_terminal`

- first_block_terminal_eq
- d1_exclusion, d2_exclusion
- d3_family_big_weight_excluded
- dge4_e2_a_ge1_excluded
- dge4_e3_j17_t1_excluded
- dge4_e3_j17_t2_delta3_excluded
- orbitSegmentWord, orbitSegmentWord_equation
- fullOrbitStep_mul_eq
- fullOrbitIter_not_dvd_five
- candidateRj_mod_five

29.3 UnifiedCoreAudit.lean

- All36_20PremisesNoHge
- blockB_bound_of_no_hge
- rj0_ge_of_size_conditions_no_hge
- terminal_bound_iff_yStar_ne_zero
- unified_core_final_no_hge — pending

29.4 CycleBridge.lean and DwdbDiv.lean

- cycleWord, cycleWord_step_exact
- cycleWord_take_eq, cycleWord_getI_eq
- cycleWord_prefix_orbit_eq
- CycleQb8Input
- cycleQb8Input_P_ge_two
- cycleQb8Input_exists_rise_index
- cycleQb8Input_exists_c3_index
- cycleQb8Input_exists_block_head_mod_five
- wordOrbit_take_succ
- rise_step_next_mod_five
- rise_block_balance
- failure_window_contradicts_window_corrected
- windowBoundToNoCycle_of_failureWindowExistence
- DwdbDiv.dwdbDivFinalAssembly

29.5 WordWindow.lean

`WordWindow.lean` defines the string-flow vocabulary:

- `StringFlow.Word.wordA`: the word molecule;
- `StringFlow.Word.wordOrbit`: the word orbit;
- `StringFlow.Word.wordValid`: word validity;
- `StringFlow.wordWeight`: the prefix-weight function;
- the exact word-orbit identity

$$2^{W(w)}\,\mathrm{wordOrbit}(w, x_0) = 5^{\mathrm{len}(w)}\,x_0 + A(w),$$

  recorded in the file's identity lemmas.

The file also hosts the elementary valuation facts `n_eq_two_pow_mul_oddPart`, `oddPart_odd_of_pos`, `twoValuation_mul_two_pow_eq`, `fiveXPlusOneStep_le_div_two`, and `c3Step_lt_of_pos`.

29.6 Pmi.lean

`Pmi.lean` hosts the PMI and PMI-B algebra:

- `Pmi.aTotal5`: the cleared PMI sum;
- `Pmi.aTotal`: the algebraic prefix sum;
- `Pmi.pmi_algebraic`: the cycle form $\mathrm{aTotal5}(P, t) = 5m(2^T - 5^P)$;
- `pmi_b_count`: the PMI-B counting bound;
- `pmi_b_no_bad_prefix`: the small-budget corollary;
- `cycleWord_margin_balance` and `cycleWord_margin_balance_strict`: the margin balance.

29.7 BlockAutomaton.lean

`BlockAutomaton.lean` hosts the corrected window definitions:

- `t1WindowBoundCorrected`;
- `t2WindowBoundCorrected`;
- `decisiveWindowValuationBoundCorrected`;
- the conditional interfaces `decisiveWindowValuationBoundCorrected_of_unified_core` and `failureWindowExistenceOfUnifiedCore`.

29.8 FinalStatement.lean

`FinalStatement.lean` hosts the top-level theorem objects:

- `fiveXPlusOneStep`;
- `fiveXPlusOneOrbit`;
- `IsUnboundedOrbit`;
- `OrbitCycle`;
- `unbounded_of_no_cycle`;
- `five_x_plus_one_diverges_at_7`, pending.

29.9 Main-chain status summary

Statement	Status
Finite-prefix expansion and first-big-step facts	compiled
`candidate_first_big_step_weight_ne_three` and `candidate_first_big_step_weight_ge_five`	compiled
`reset_head_predecessor`	compiled
`candidateX_of_reset_and_terminal`	compiled
`first_block_terminal_eq`	compiled
`d1_exclusion`, `d2_exclusion`	compiled
`d3_family_big_weight_excluded`	compiled
`dge4_e2_a_ge1_excluded`	compiled
`dge4_e3_j17_t1_excluded`, `dge4_e3_j17_t2_delta3_excluded`	compiled
`orbitSegmentWord_equation`	compiled
`All36_20PremisesNoHge` (definition)	compiled
`blockB_bound_of_no_hge`	compiled
`rj0_ge_of_size_conditions_no_hge`	compiled
`terminal_bound_iff_yStar_ne_zero`	compiled
`unified_core_final_no_hge`	pending
`CycleQb8Input` and extraction lemmas	compiled
`failure_window_contradicts_window_corrected`	compiled
`windowBoundToNoCycle_of_failureWindowExistence`	compiled (conditional)
`dwdbDivFinalAssembly`	compiled (conditional)

The statuses are the current audit state; they are rechecked whenever the repository changes.

29.10 AxiomAudit

The audit of the bridge theorems currently returns only

```
[propext, Classical.choice, Quot.sound]
```

for the theorems that use choice, and

```
[propext, Quot.sound]
```

for the theorems that do not. No custom axiom is used.

Theorem	Axioms
`no_cycle_of_window_bound`	`[propext, Quot.sound]`
`failure_window_contradicts_window_corrected`	`[propext, Quot.sound]`
`five_x_plus_one_diverges_at_7_of_window_bound`	`[propext, Classical.choice, Quot.sound]`
`DwdbDiv.dwdbDivFinalAssembly`	`[propext, Classical.choice, Quot.sound]`

29.11 Status convention

- Math: proven.
- Lean: compiled without `sorry`.
- Lean: pending means the Lean statement still contains `sorry` or is not yet assembled.

29.12 Per-file status ledger

File	Status
`WordWindow.lean`	definitions compiled
`Pmi.lean`	PMI and PMI-B compiled
`FinitePrefix.lean`	finite base compiled
`UnifiedCoreBridge.lean`	bridges compiled
`UnifiedCoreAudit.lean`	one pending `sorry`
`CycleBridge.lean`	compiled (conditional)
`DwdbDiv.lean`	compiled (conditional)
`FinalStatement.lean`	assembly target pending
`AxiomAudit.lean`	audit output recorded

29.13 Key statement shapes

The bridge lemmas have the following ASCII statement shapes.

```
theorem reset_head_predecessor
    (s0 j k t delta rj x : Nat)
    (hj : 1 <= j)
    (hres : ResetHeadEq s0 j k t delta rj)
    (hrj : rj = (5 * x + 1) / 2 ^ t)
    (hdiv : (5 * x + 1) % 2 ^ t = 0) :
    x = 5 ^ k * s0 + delta * 5 ^ (j - 1) - 1

theorem first_block_terminal_eq
    (e g_prev g r : Nat)
    (hg : 5 * g_prev + 1 = 2 ^ e * g)
    (hr : r = (5 * g_prev + 1) / 2)
    (he : 1 <= e) :
    r = 2 ^ (e - 1) * g

theorem d1_exclusion
    (j e g delta a : Nat)
```

```
    (hj : 3 <= j) (hgpos : 0 < g)
    (hg : g < 5 ^ (j - 1) / 4)
    (hdelta : delta = 1 or delta = 3)
    (he : 2 <= e)
    (hy : 5 * g + 1 = 2 ^ (1 + 4 * a) * candidateX j e g delta) :
    False

theorem d2_exclusion
    (j e g delta u1 : Nat)
    (hj : 3 <= j)
    (hg : g < 5 ^ (j - 1) / 2 ^ (e - 1))
    (hdelta : delta = 1 or delta = 3)
    (he : 2 <= e)
    (hu1 : u1 = 1 or u1 = 2)
    (hseg : 2 ^ (u1 + e) * g + 2 ^ (u1 + 1) * delta * 5 ^ (j - 1)
      = 5 + 2 ^ u1 + 25 * g) :
    False
```

29.14 Additional statement shapes

The cycle bridge adds:

```
theorem cycleWord_cycle_equation (c p : Nat)
    (hclosed : wordOrbit (cycleWord c p) (Orbit 7 c) = Orbit 7 c) :
    2 ^ wordWeight (cycleWord c p) * Orbit 7 c
      = 5 ^ p * Orbit 7 c + wordA (cycleWord c p)

theorem cycleWord_step_exact (c p k : Nat) (hk : k < p) :
    twoValuation
      (5 * wordOrbit ((cycleWord c p).take k) (Orbit 7 c) + 1)
      = (cycleWord c p).getI k

theorem failure_window_contradicts_window_corrected
    (hwin : decisiveWindowValuationBoundCorrected)
    {m S P : Nat} {w rise c3 : List Nat}
    {j k0 t delta s : Nat}
    (fw : FailureWindow m S P w rise c3 j k0 t delta s) :
    False
```

29.15 FinitePrefix statement shapes

The finite base is recorded by:

```
theorem fullOrbitIter_prefix_expand :
    fullOrbitIter 0 = 7 /\
    fullOrbitIter 1 = 9 /\
    fullOrbitIter 2 = 23 /\
    fullOrbitIter 3 = 29 /\
    fullOrbitIter 4 = 73 /\
```

```
    fullOrbitIter 5 = 183 /\
    fullOrbitIter 6 = 229 /\
    fullOrbitIter 7 = 573 /\
    fullOrbitIter 8 = 1433 /\
    fullOrbitIter 9 = 3583 /\
    fullOrbitIter 10 = 4479 /\
    fullOrbitIter 11 = 5599 /\
    fullOrbitIter 12 = 6999 /\
    fullOrbitIter 13 = 8749 /\
    fullOrbitIter 14 = 21873 /\
    fullOrbitIter 15 = 54683 /\
    fullOrbitIter 16 = 34177

theorem fullOrbit_first_t_ge3_is_exactly_3 :
    (forall n : Nat, n < 15 ->
      twoValuation (5 * fullOrbitIter n + 1) <= 2) /\
    twoValuation (5 * fullOrbitIter 15 + 1) = 3

theorem first_big_step_unique (n : Nat)
    (hsmall : forall m : Nat, m < n ->
      twoValuation (5 * fullOrbitIter m + 1) <= 2)
    (hbig : 3 <= twoValuation (5 * fullOrbitIter n + 1)) :
    n = 15 /\ twoValuation (5 * fullOrbitIter n + 1) = 3
```

29.16 UnifiedCoreAudit statement shapes

```
theorem terminal_bound_iff_yStar_ne_zero
    (L H_s s r_s : Nat) (hH : 2 <= H_s) :
    twoValuation (5 ^ (L + 3) * wTerminal L r_s + 1) <= H_s - 2 <->
      yStar L H_s s r_s != 0

theorem rj0_ge_of_size_conditions_no_hge
    (j Wp Wj q Aj A_s s W_s r_s L H_s : Nat)
    (weight : Nat -> Nat)
    (hPrem : All36_20PremisesNoHge
       j Wp Wj q Aj A_s s W_s r_s L H_s weight)
    (hH : 1 <= H_s)
    (hBase2 : 3 * 5 ^ s
      + (3 * 5 ^ (s - j) - 2 * 4 ^ (s - j))
      <= 2 ^ (W_s - Wj + L + 4) * uResidue L (H_s - 1))
    (hPow3 : 3 * 5 ^ s
      + (3 * 5 ^ (s - j) - 2 * 4 ^ (s - j))
      <= 2 ^ (W_s - Wj + L + H_s + 3)) :
    5 ^ j <= rj0 j Wp Wj q Aj A_s s W_s r_s L H_s weight
```

29.17 Reading order

1. `WordWindow.lean`: `wordA`, `wordOrbit`, `wordValid`;

2. `Pmi.lean`: PMI and PMI-B algebra;

3. `FinitePrefix.lean`: the explicit finite base;

4. `UnifiedCoreBridge.lean`: candidate parameterisation and segment exclusions;

5. `UnifiedCoreAudit.lean`: CRT, terminal residue, and the final pending statement;

6. `CycleBridge.lean`: corrected window and no-cycle bridge;

7. `DwdbDiv.lean`: final divergence assembly.

29.18 Acceptance criteria

- no `sorry` in any theorem consumed by the main chain;
- no custom axiom; only `propext`, `Classical.choice`, and `Quot.sound` appear;
- no `native_decide` in the main chain; the finite base is expanded by rewriting;
- `unified_core_final_no_hge` is recorded as pending and is not cited as closed.

30 Dependency graph and status ledger

The dependency graph is a directed acyclic graph whose nodes are mathematical lemmas and Lean theorems.

30.1 Main dependency chain

1. Finite base: `FinitePrefix.lean`.

2. Candidate and segment bridges: `UnifiedCoreBridge.lean`.

3. Unified core: `UnifiedCoreAudit.lean`.

4. Cycle bridge: `CycleBridge.lean`.

5. Final assembly: `DwdbDiv.lean`.

In topological order, the individual statements are:

1. explicit orbit values and first-big-step facts;

2. candidate parameterisation and terminal equations;

3. orbit segment equations and $d = 1, d = 2$ exclusions;

4. $d = 3$ and $d \geq 4$ family exclusions;

5. CRT candidate, terminal residue, and size conditions;

6. the unified-core final inequality (pending);

7. cycle-word extraction and the QB-8 split;

8. corrected-window projection and failure-window contradiction;
9. window-to-no-cycle bridge;
10. final divergence assembly.

Lemma	File	Status
fullOrbitIter_prefix_expand	FinitePrefix.lean	compiled
reset_head_predecessor	UnifiedCoreBridge.lean	compiled
d2_exclusion	UnifiedCoreBridge.lean	compiled
unified_core_final_no_hge	UnifiedCoreAudit.lean	pending
windowBoundToNoCycle_of_failureWindowExistence	CycleBridge.lean	compiled (conditional)
dwdbDivFinalAssembly	DwdbDiv.lean	compiled (conditional)

30.2 Dependency edges

Statement	Direct dependencies	Status
orbit_cycle_imp_cycle_qb8_input	orbit_cycle_imp_cycle_word_valid_closed, cycleWord_*	compiled
orbitSegmentWord_candidate_equation	orbitSegmentWord_equation, candidateX	compiled
d2_exclusion_of_orbit	d2_segment_equation, d2_exclusion	compiled
unified_core_final_no_hge	rj0_ge_of_size_conditions_no_hge, terminal_bound_iff_yStar_ne_zero	pending
failure_window_contradicts_window_corrected	FailureWindow,	
decisiveWindowValuation		
BoundCorrected	compiled	
windowBoundToNoCycle_of_failureWindowExistence	failureWindowExistence, failure_window_contradicts_window_corrected	compiled (conditional)
dwdbDivFinalAssembly	windowBoundToNoCycle, unbounded_of_no_cycle	compiled (conditional)

The table records the direct edges only. Transitive dependencies are obtained by following the corresponding file imports.

30.3 File import graph

The main files import as follows:

```
FinitePrefix.lean
  imports S6AuditStage1

UnifiedCoreBridge.lean
  imports FinitePrefix, UnifiedCoreAudit

UnifiedCoreAudit.lean
  imports S6Audit, S6AuditStage1, LteMacro

CycleBridge.lean
  imports BlockAutomaton, FinalStatement, S6AuditStage1,
          PhOne, SurvExAudit, TdOReal
```

```
DwdbDiv.lean
  imports CycleBridge, UnifiedCoreAudit

FinalStatement.lean
  imports S6Audit, FBeta
```

The import graph is read from the file headers; it is updated whenever the Lean project re-imports.

30.4 Verification commands

The manual uses the same verification commands as the main paper:

```
lake build CycleBridge
lake build DwdbDiv
```

For a status change, also re-run the axiom audit and check that `UnifiedCoreAudit.lean` contains exactly one `sorry`, namely `unified_core_final_no_hge`.

30.5 Ledger locations

- `docs/lean_assembly_plan.md`
- `docs/cycle_bridge_window_to_no_cycle.md`
- `docs/dwdb_div_assembly.md`

The ledger is authoritative for status. A Lean statement marked pending must not be cited as closed in the paper or in the manual.

30.6 Open status and next steps

Two statements remain open:

- `UnifiedCoreAudit.unified_core_final_no_hge`: the unique Lean `sorry` in the unified-core audit;
- `FinalStatement.five_x_plus_one_diverges_at_7`: the final assembly target.

When the first statement is closed, the dependency-edge table and all status tables must be updated before the final theorem is promoted. The paper does not claim either statement as closed.

31 Finite-prefix base and explicit orbit expansion

This chapter records the finite-prefix base in full. The base is the only finite data used by the proof. Every entry is an exact arithmetic identity, expanded by rewriting in Lean rather than by `native_decide`.

31.1 The 17 states

The accelerated $5x+1$ orbit of 7 begins with

$$7, 9, 23, 29, 73, 183, 229, 573, 1433, 3583, 4479, 5599, 6999, 8749, 21873, 54683, 34177.$$

There are seventeen entries, indexed by $n = 0, \ldots, 16$. The proof never needs a later state.

31.2 Exact step table

Each row below records the exact equation

$$5\,\mathrm{Orbit}_7(n) + 1 = 2^{t_n}\,\mathrm{Orbit}_7(n+1),$$

with $t_n = v_2(5\,\mathrm{Orbit}_7(n) + 1)$.

n	$\mathrm{Orbit}_7(n)$	$5\mathrm{Orbit}_7(n)+1$	t_n	$\mathrm{Orbit}_7(n+1)$
0	7	36	2	9
1	9	46	1	23
2	23	116	2	29
3	29	146	1	73
4	73	366	1	183
5	183	916	2	229
6	229	1146	1	573
7	573	2866	1	1433
8	1433	7166	1	3583
9	3583	17916	2	4479
10	4479	22396	2	5599
11	5599	27996	2	6999
12	6999	34996	2	8749
13	8749	43746	1	21873
14	21873	109366	1	54683
15	54683	273416	3	34177

The divisibility checks are literal:

$$\begin{aligned}
36 &= 4 \cdot 9, & 46 &= 2 \cdot 23, & 116 &= 4 \cdot 29,\\
146 &= 2 \cdot 73, & 366 &= 2 \cdot 183, & 916 &= 4 \cdot 229,\\
1146 &= 2 \cdot 573, & 2866 &= 2 \cdot 1433, & 7166 &= 2 \cdot 3583,\\
17916 &= 4 \cdot 4479, & 22396 &= 4 \cdot 5599, & 27996 &= 4 \cdot 6999,\\
34996 &= 4 \cdot 8749, & 43746 &= 2 \cdot 21873, & 109366 &= 2 \cdot 54683,\\
273416 &= 8 \cdot 34177. & & & &
\end{aligned}$$

31.3 Step weights

Reading the third column from the table gives the exact weight word

$$t_0, \ldots, t_{15} = 2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1, 1, 3.$$

The first fifteen weights, for $n < 15$, are all at most 2. The first weight of size at least 3 occurs at $n = 15$ and equals 3 exactly.

Lemma 31.1 (First big step)**.** *For every $n < 15$, $t_n \le 2$, and*

$$t_{15} = 3.$$

Consequently the first step of weight at least 3 is unique and is the step from depth 15 to depth 16.

Proof. The first statement is the inspection of the table above. The uniqueness follows because all earlier weights are at most 2 and the weight at depth 15 is 3. □

31.4 Prefix ledger

Let W_n be the prefix weight after n steps and let A_n be the word molecule of the exact prefix. The table below repeats the main-paper ledger with the exact prefix word displayed.

n	prefix word	t_n	W_n	A_n
0	[]	–	0	0
1	[2]	2	2	1
2	[2, 1]	1	3	9
3	[2, 1, 2]	2	5	53
4	[2, 1, 2, 1]	1	6	297
5	[2, 1, 2, 1, 1]	1	7	1549
6	[2, 1, 2, 1, 1, 2]	2	9	7873
7	[2, 1, 2, 1, 1, 2, 1]	1	10	39877
8	[2, 1, 2, 1, 1, 2, 1, 1]	1	11	200409
9	[2, 1, 2, 1, 1, 2, 1, 1, 1]	1	12	1004093
10	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2]	2	14	5024561
11	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2]	2	16	25139189
12	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2]	2	18	125761481
13	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2]	2	20	629069549
14	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1]	1	21	3146396321
15	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1, 1]	1	22	15734078757
16	[2, 1, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 2, 1, 1, 3]	3	25	78674588089

Every row is an instance of the exact word-orbit identity

$$2^{W_n}\,\mathrm{Orbit}_7(n) = 5^n \cdot 7 + A_n.$$

For example, the $n = 16$ row reads

$$2^{25} \cdot 34177 = 5^{16} \cdot 7 + 78674588089.$$

This identity is a theorem in the string-flow layer; it is not a finite regression fact and does not use `native_decide`.

31.5 Why explicit rewriting

The finite base is deliberately expanded by explicit rewriting instead of decision procedures. A `native_decide` certificate would move the arithmetic into the code-generation trust boundary. The explicit form keeps every multiplication and division visible and lets `lake build` check the chain without any hidden computation. The Lean statement `fullOrbitIter_prefix_expand` is a conjunction of equalities of the form

```
fullOrbitIter 0 = 7 /\ fullOrbitIter 1 = 9 /\
fullOrbitIter 2 = 23 /\ ... /\ fullOrbitIter 16 = 34177
```

31.6 Use in the segment exclusions

The finite base is used in exactly two ways.

First, the $d = 3$ survivor produces a candidate first big step of weight 6 at depth $j + 4$. By the finite base, the first big step of the orbit of 7 occurs at depth 15 and has weight 3, not 6. The Lean statement `candidate_first_big_step_weight_ne_three` turns this into a contradiction.

Second, the $d \geq 4$ branches produce a candidate first big step of weight at least 5, or reduce to the $e = 3$, $j = 17$ residue exclusions. A first big step of weight at least 5 cannot occur at depth 15, where the true weight is 3, and cannot occur before it, where all weights are at most 2; if it occurred later, the true depth-15 step would already be first. The Lean statement `candidate_first_big_step_weight_ge_five` records this case.

31.7 First big step uniqueness

The uniqueness lemma has the following Lean shape.

```
theorem first_big_step_unique (n : Nat)
    (hsmall : forall m : Nat, m < n ->
      twoValuation (5 * fullOrbitIter m + 1) <= 2)
    (hbig : 3 <= twoValuation (5 * fullOrbitIter n + 1)) :
    n = 15 /\ twoValuation (5 * fullOrbitIter n + 1) = 3
```

31.8 Related finite facts

- `fullOrbitIter_prefix_expand`: the seventeen exact values.
- `fullOrbit_prefix_step_weights`: the exact weights for $n < 16$.
- `fullOrbit_first_t_ge3_is_exactly_3`: the first big step is at depth 15 with weight 3.
- `first_big_step_unique`: uniqueness of the first big step.

These four facts are the only output of the finite-prefix chapter that the unified core consumes.

31.9 Status

Statement	Status
exact 17-state expansion	Lean: compiled
exact step weights	Lean: compiled
first big step at depth 15 with weight 3	Lean: compiled
first big step unique	Lean: compiled
no `native_decide` in the chain	verified by inspection

32 CRT candidate and terminal residue

This chapter records the CRT layer of the unified core. The layer reduces the final valuation inequality to exact modular arithmetic and two size conditions. The mathematical derivation is complete; the Lean statement `unified_core_final_no_hge` that assembles the last step remains pending.

32.1 Setup

Let j be the reset depth, let s be the terminal depth, and write

$$n = s - j, \qquad \Delta = W_s - W_j.$$

Let L be the length of the block tail and let B be the block molecule of the tail word. The tail is a valid word from the block head $r_{j,0}$ to the terminal state r_s, so the exact word-orbit identity gives

$$2^{\Delta}\, r_s = 5^n\, r_{j,0} + B.$$

32.2 The terminal odd part

The terminal odd part is

$$w_{\mathrm{term}} = \frac{3r_s + 1}{2^{L+4}}.$$

Multiplying the tail equation by $3 \cdot 2^{\Delta}$ and adding 2^{Δ} gives

$$2^{\Delta+L+4}\, w_{\mathrm{term}} = 2^{\Delta}(3r_s + 1) = 3 \cdot 5^{n}\, r_{j,0} + 3B + 2^{\Delta}.$$

This identity is the exact algebra connecting the orbit equation to the valuation inequality. It is the identity recorded by `rj0_spec_eq`.

32.3 The valuation obstruction

The target inequality is

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \le H_s - 2.$$

Its negation is

$$v_2\big(5^{L+3} w_{\mathrm{term}} + 1\big) \ge H_s - 1,$$

which holds exactly when

$$5^{L+3} w_{\mathrm{term}} + 1 \equiv 0 \pmod{2^{H_s-1}}.$$

Since 5 is odd, $5^{-(L+3)}$ exists modulo 2^{H_s-1}, and the negation is exactly

$$w_{\mathrm{term}} \equiv -5^{-(L+3)} \pmod{2^{H_s-1}}.$$

Let u be the least nonnegative residue of $-5^{-(L+3)}$ modulo 2^{H_s-1}.

32.4 The CRT equation

Assume, toward the contradiction, that the valuation inequality fails. Then $w_{\mathrm{term}} \equiv u \pmod{2^{H_s-1}}$, so

$$w_{\mathrm{term}} = u + m'\, 2^{H_s-1}$$

for some integer $m' \ge 0$. Substituting into the terminal identity gives the exact CRT equation

$$3 \cdot 5^{n}\, r_{j,0} + 3B + 2^{\Delta} = 2^{\Delta+L+4}\left(u + m'\, 2^{H_s-1}\right).$$

The CRT candidate $r_{j,0}$ is the least nonnegative solution of this equation. The equation is linear in $r_{j,0}$, so the candidate is well defined; its explicit form is used in `rj0_spec_eq`.

32.5 The terminal residue y^*

The residue

$$y^* = -\left(w_{\mathrm{term}} + 5^{-(L+3)}\right)(3 \cdot 5^{s})^{-1} \pmod{2^{H_s-1}}$$

is the normalised failure residue. Because $3 \cdot 5^{s}$ is odd, its inverse modulo 2^{H_s-1} exists. By construction,

$$y^* = 0 \iff w_{\mathrm{term}} \equiv -5^{-(L+3)} \pmod{2^{H_s-1}},$$

so the valuation inequality is equivalent to $y^* \neq 0$. The Lean statement is `terminal_bound_iff_yStar_ne_zero`, compiled.

32.6 The two size conditions

The first size condition controls the correction term:

$$3B + 2^{\Delta} \le 3 \cdot 5^n - 2 \cdot 4^n.$$

The second controls the leading term:

$$3 \cdot 5^s + (3 \cdot 5^n - 2 \cdot 4^n) \le 2^{\Delta+L+4}\, u.$$

Both are consequences of the no-$H_{\ge}$ premises; the Lean statement `blockB_bound_of_no_hge` packages the first one.

32.7 From the size conditions to $r_{j,0} \ge 5^j$

From the CRT equation,

$$3 \cdot 5^n\, r_{j,0} = 2^{\Delta+L+4} \left(u + m' 2^{H_s - 1}\right) - 3B - 2^{\Delta}.$$

The right-hand side is at least

$$2^{\Delta+L+4}\, u - (3 \cdot 5^n - 2 \cdot 4^n),$$

using the first size condition. The second size condition bounds this below by $3 \cdot 5^s$. Therefore

$$3 \cdot 5^n\, r_{j,0} \ge 3 \cdot 5^s,$$

so

$$r_{j,0} \ge 5^{s-n} = 5^j.$$

This is the content of `rj0_ge_of_size_conditions_no_hge`, compiled.

32.8 Role of the block-head interval

The no-$H_{\ge}$ premises also carry interval constraints on the block head. The final unified-core statement must compare $r_{j,0} \ge 5^j$ with those constraints. The comparison is not an independent analytic estimate: it is the remaining assembly step `unified_core_final_no_hge`, which is the single pending `sorry` in the unified-core audit.

32.9 Worked modular checks

The $d = 2$ exclusion uses two modular facts. Modulo 17,

$$5^3 = 125 \equiv 6 \pmod{17},$$

and the order of 5 modulo 17 is 16, so the congruence $5^{j-1} \equiv 6$ fixes $j - 1 \equiv 3 \pmod{16}$. Modulo 256,

$$5^7 = 78125 \equiv 45 \pmod{256},$$

and the order of 5 modulo 256 is 64, so $5^{j-1} \equiv 45$ fixes $j - 1 \equiv 7 \pmod{64}$. The two congruences contradict each other modulo 16.

The $d = 3$ family has denominator 109. The consistency check is carried out by repeated squaring modulo 109:

k	$5^k \bmod 109$	note
1	5	
2	25	
4	80	$625 - 5 \cdot 109$
8	78	$6400 - 58 \cdot 109$
16	89	$6084 - 55 \cdot 109$
32	73	$7921 - 72 \cdot 109$
64	97	$5329 - 48 \cdot 109$
128	35	$9409 - 86 \cdot 109$
256	26	$1225 - 11 \cdot 109$
512	22	$676 - 6 \cdot 109$

Since

$$923 = 512 + 256 + 128 + 16 + 8 + 2 + 1,$$

the product is

$$22 \cdot 26 \cdot 35 \cdot 89 \cdot 78 \cdot 25 \cdot 5 \equiv 27 \cdot 35 \cdot 89 \cdot 78 \cdot 25 \cdot 5 \equiv 73 \pmod{109}.$$

Hence $5^{923} \equiv 73 \pmod{109}$, and

$$8 \cdot 73 = 584 \equiv 39 \pmod{109},$$

which is exactly the divisibility required by

$$g = \frac{8 \cdot 5^{923} - 39}{109}$$

for the representative $j - 1 = 923$ of the residue class $j - 1 \equiv 923 \pmod{1728}$.

32.10 Lean statements

- `All36_20PremisesNoHge`: the record of the no-$H_{\geq}$ premises.
- `blockB_bound_of_no_hge`: the block-tail bound.
- `rj0_spec_eq`: the CRT equation and candidate.
- `terminal_bound_iff_yStar_ne_zero`: the valuation equivalence.
- `rj0_ge_of_size_conditions_no_hge`: the size-condition lower bound.
- `unified_core_final_no_hge`: pending.

32.11 Status ledger

Statement	Status
CRT equation and candidate definition	Lean: compiled
valuation iff $y^* \neq 0$	Lean: compiled
size-condition lower bound $r_{j,0} \geq 5^j$	Lean: compiled
assembly to the final valuation inequality	Lean: pending

The paper does not claim that the pending assembly is closed.

33 Lean bridge interfaces

This chapter records the interface graph of the Lean bridge. The graph is the formal skeleton of the paper: it shows exactly which statements are compiled, which are conditional, and which remain open.

33.1 Conventions

- **compiled**: the theorem compiles in the pinned Lean environment without `sorry`.
- **compiled (conditional)**: the theorem compiles, but one of its inputs is an open proposition such as `unifiedCoreClosed` or `failureWindowExistence`.
- **pending**: the theorem contains `sorry` or has not yet been assembled.

33.2 Interface map

```
FinitePrefix.lean (compiled)
  -> UnifiedCoreBridge.lean (compiled)
  -> UnifiedCoreAudit.lean (one pending: unified_core_final_no_hge)
  -> unifiedCoreClosed (open input)
  -> decisiveWindowValuationBoundCorrected_of_unified_core (compiled*)
  -> failureWindowExistenceOfUnifiedCore (compiled*)
  -> CycleBridge.windowBoundToNoCycle_of_failureWindowExistence
       (compiled*, conditional)
  -> DwdbDiv.dwdbDivFinalAssembly (compiled*, conditional)
  -> FinalStatement.five_x_plus_one_diverges_at_7 (pending)
```

The arrows are implications in the theorem statements, not simulation steps. Each conditional node compiles because it is written as a function of its open input.

33.3 Conditional interfaces

Interface	Consumes	Status
`decisiveWindowValuationBoundCorrected_of_unified_core`	`unifiedCoreClosed`	compiled*
`failureWindowExistenceOfUnifiedCore`	`unifiedCoreClosed`	compiled*
`windowBoundToNoCycle_of_failureWindowExistence`	`failureWindowExistence`	compiled*
`dwdbDivFinalAssembly`	`windowBoundToNoCycle`	compiled*
`five_x_plus_one_diverges_at_7`	`dwdbDivFinalAssembly`	pending

33.4 What is already closed

The compiled layer includes:

- the finite-prefix expansion and first-big-step facts;
- the candidate parameterisation and reset-head predecessor;
- the segment equations and the $d = 1, d = 2, d = 3, d \geq 4$ exclusions;

- the CRT equation, the terminal residue equivalence, and the size-condition lower bound;
- the cycle-word extraction and QB-8 split;
- the failure-window contradiction lemma;
- the window-to-no-cycle bridge, conditional on the open inputs.

33.5 What remains open

Exactly two statements are open:

Statement	Status
`UnifiedCoreAudit.unified_core_final_no_hge`	pending
`FinalStatement.five_x_plus_one_diverges_at_7`	pending

The second statement is an assembly target: once the first statement is closed, the compiled conditional interfaces produce the final theorem.

33.6 Axiom audit

The audit of the bridge theorems returns only

`[propext, Classical.choice, Quot.sound]`

for theorems that use choice, and

`[propext, Quot.sound]`

for theorems that do not. The audit runs `#print axioms` on the actual theorems and compares the result with the allowlist. No custom axiom is used.

Theorem	Axioms
`no_cycle_of_window_bound`	`[propext, Quot.sound]`
`failure_window_contradicts_window_corrected`	`[propext, Quot.sound]`
`five_x_plus_one_diverges_at_7_of_window_bound`	`[propext, Classical.choice, Quot.sound]`
`DwdbDiv.dwdbDivFinalAssembly`	`[propext, Classical.choice, Quot.sound]`

33.7 Acceptance criteria

1. No `sorry` in any theorem consumed by the main chain.
2. No custom `axiom`; only `propext`, `Classical.choice`, and `Quot.sound` are allowed.
3. No `native_decide` in the main chain; the finite base is expanded explicitly.
4. `lake build CycleBridge` and `lake build DwdbDiv` pass.
5. `unified_core_final_no_hge` is promoted only after its `sorry` is replaced by a proof.
6. The final theorem name and statement are exactly

```
FinalStatement.five_x_plus_one_diverges_at_7 :
  IsUnboundedOrbit 7
```

33.8 Update workflow

Whenever the Lean repository changes, the following steps are run:

1. rebuild the affected targets;
2. re-run the axiom audit;
3. update the dependency tables in this chapter and in the main paper;
4. update the status ledger in `STATUS.md`;
5. recompile the paper and the manual;
6. do not promote any pending statement before its ledger entry is changed.

34 Manual chapters

1. String-flow vocabulary and exact word identities.
2. PMI and PMI-B budgets.
3. Block calculus and reset equations.
4. Corrected reachability and the decisive window bound.
5. Unified core: candidate parameterisation and segment exclusions.
6. Finite-prefix base and explicit orbit expansion.
7. CRT candidate and terminal residue.
8. Lean bridge interfaces.
9. Divergence bridge and failure windows.
10. Arithmetic counterexample and corrected window.
11. Failure window construction.
12. Worked computations ledger.
13. Theorem-by-theorem status matrix.
14. Genericity ledger for (a, b).
15. Cycle word algebra.
16. Corrected window bound architecture.
17. Lean build and reproducibility guide.
18. Block-head candidate exclusion and the remaining bridge.
19. Finite-prefix verification details.
20. The divergence bridge in full.

21. Rise blocks, C3 blocks, and the block-layer balance.
22. Glossary, notation, and reading guide.
23. Proof architecture and dependency diagrams.
24. Why the proof is exact.
25. Full derivation of the $d = 2$ exclusion.
26. Full derivation of the $d = 3$ exclusion.
27. Full derivation of the $d \geq 4$ exclusions.
28. Finite-prefix proof script outline.
29. The CRT layer: statement-by-statement commentary.
30. The corrected reachability predicate.
31. Worked examples of the exact ledger.
32. Submission and release checklist.
33. Lean theorem index.
34. Dependency graph and status ledger.

35 Lean theorem index

Each entry will contain:

- mathematical statement;
- Lean name and file;
- status: `Math: proven`, `Lean: compiled`, or `Lean: pending`;
- list of dependencies.

36 Status ledger

Statement	Status
`FinitePrefix` base	Lean: compiled
`UnifiedCoreBridge` segment exclusions	Lean: compiled
`UnifiedCoreAudit.unified_core_final_no_hge`	Lean: pending
`FinalStatement.five_x_plus_one_diverges_at_7`	Lean: assembly target, pending
`CycleBridge` corrected window assembly	Lean: compiled (conditional)
`DwdbDiv` final assembly	Lean: compiled (conditional)

37 Cross-reference policy

The manual cross-references the main paper by section and theorem number. It does not paste Lean files line by line; it lists definitions and theorems, with file and line regions.